CSF205:
Web Service Development

Week 6: Mid-Term Revision (and Writing Clients)

Swansea University
Prifysgol Abertawe

Department of Computer Science
Applied Software Engineering

DISCLAIMER

Attending this lecture/reading only this slide deck is not enough to revise the module

- These slides should serve as a streamlined study guide
- In-class demos and lab tasks also introduced examinable material

You might be asked to:

- Identify / explain errors, design choices and best practices
- Describe how certain technologies works
- Explain / debug code snippets

You will not be asked to:

- Recite definitions

Concerning Questions: Wording

Questions use different fonts. Fonts convey meaning:

Italics: Key module-specific terminology

`Teletype`: Tools, technologies or protocols

Bold: Actions required to answer the question

The ‘bold’ actions also convey meaning:

Identify: There are discrete factors you should mention

Describe: There are multiple concrete steps to mention

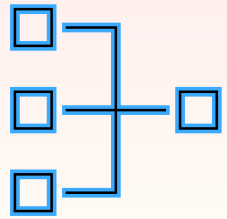
Explain: You should provide insight or analysis

Justify: When ‘identification’ needs an explanation



WEEK 1

Computer Networks and Distributed Systems



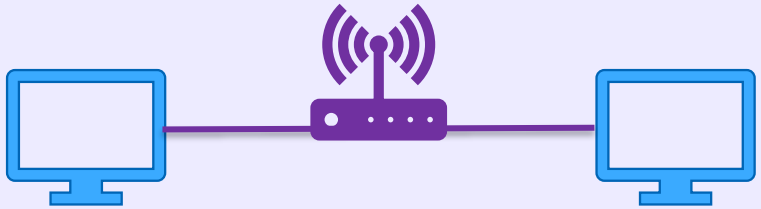
Terminology: Connection



Wired



Wireless

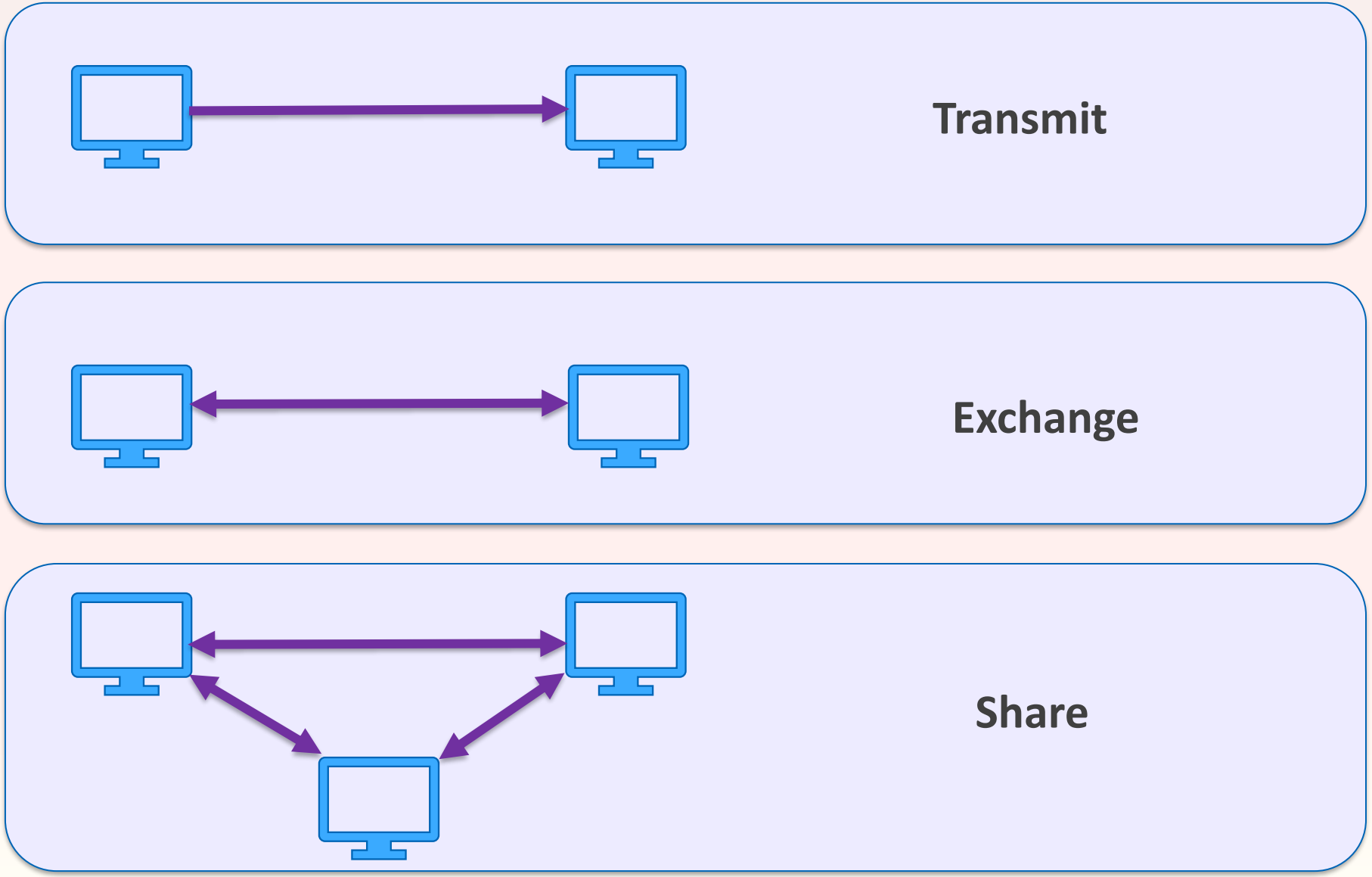


Intermediary

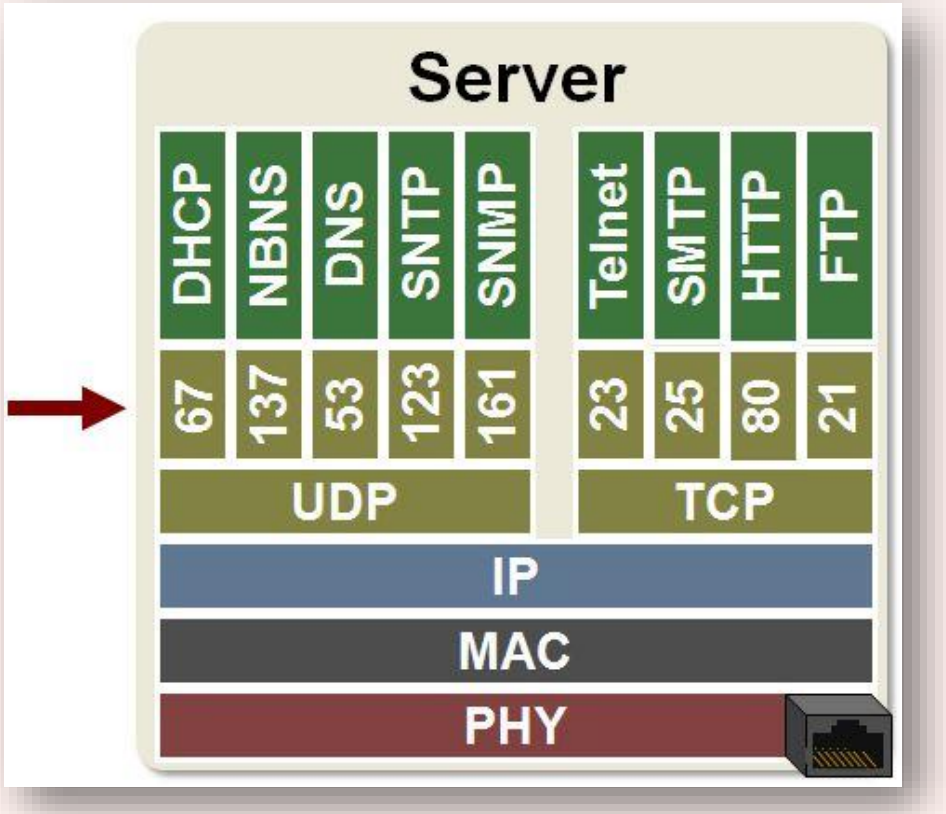


Virtually

Terminology: Communication



TCP/IP: Ports



Ports are 16-bit numbers
Range: 0-65525

- Applications are managed by processes.
- Ports identify processes on a host.
- They are virtual addresses.
- Common applications have designated ports.
- Ephemeral ports are free to use by Clients.

Note: Never use a port < 1024 unless you mean to use a specific service!

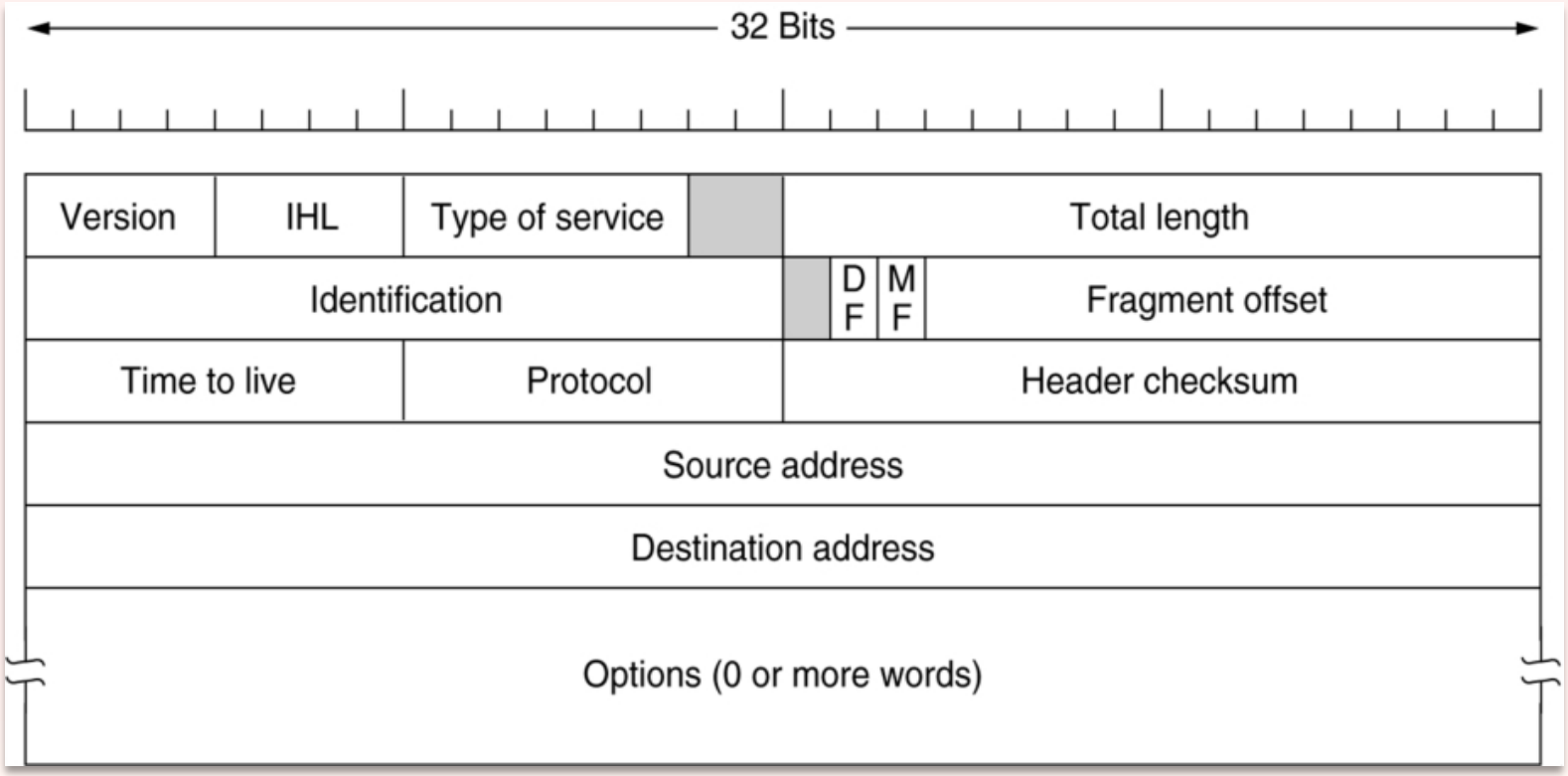
Concept Check: Sending Data

MAC, TCP/IP, Ports, DCHP and DNS provide device-device addressing.

What does the data look like?



TCP/IP: The IP Packet



TCP/IP doesn't send a stream of data.

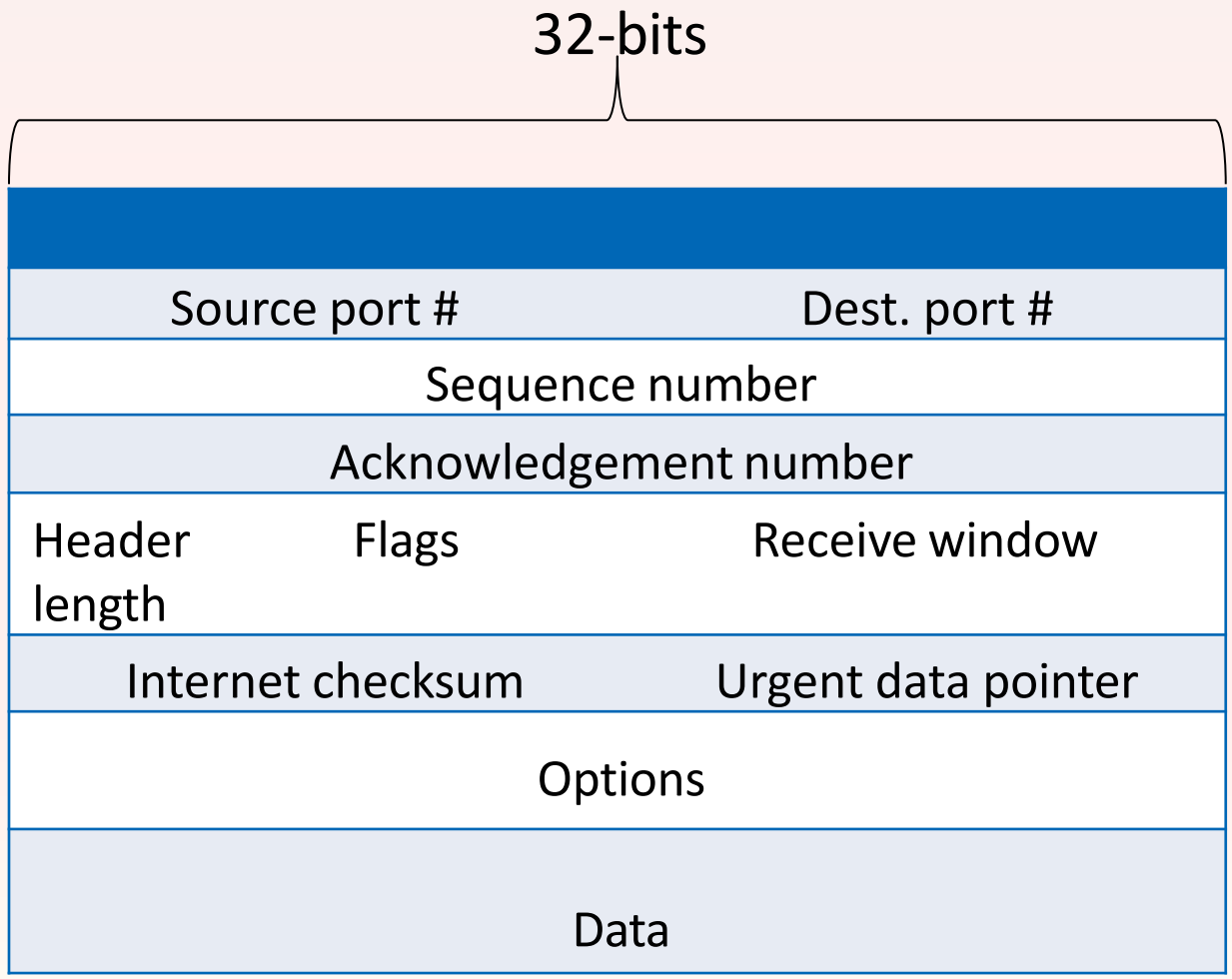
It emulates it by sending chunks of data.

These chunks are called packets.

As far as most programmers are concerned, it is sent and received as a stream.

TCP/IP: Transmission Control Protocol

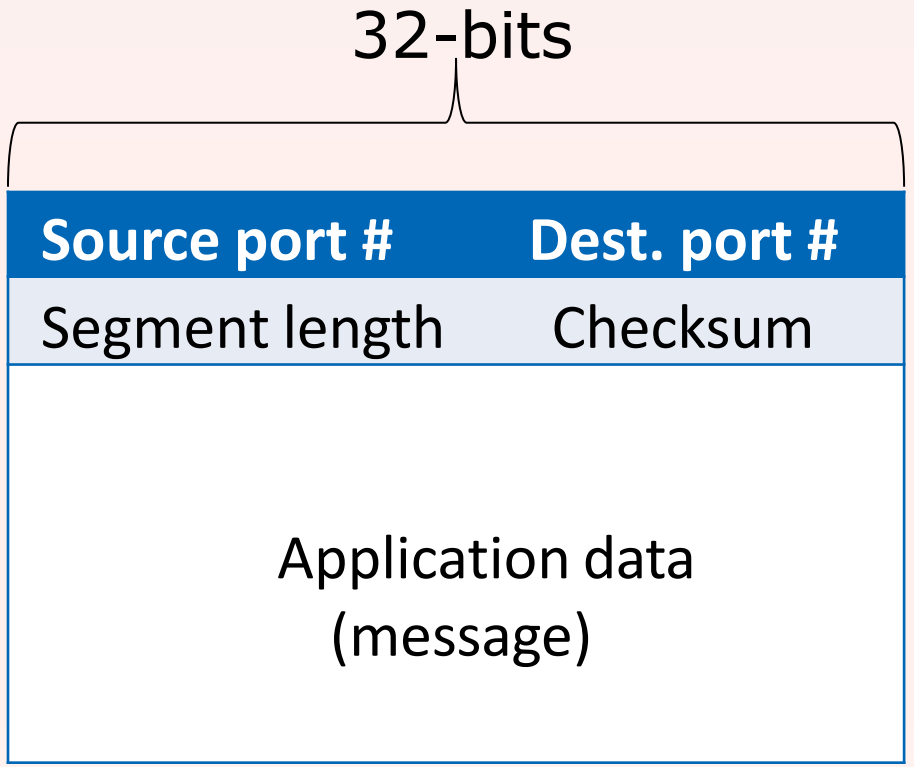
- Stateful, reliable, connection-oriented.
- Full duplex service.
- 3-way handshake to establish connection.
- TCP code can buffer bytes, rather than sending immediately.



TCP/IP: User Datagram Protocol

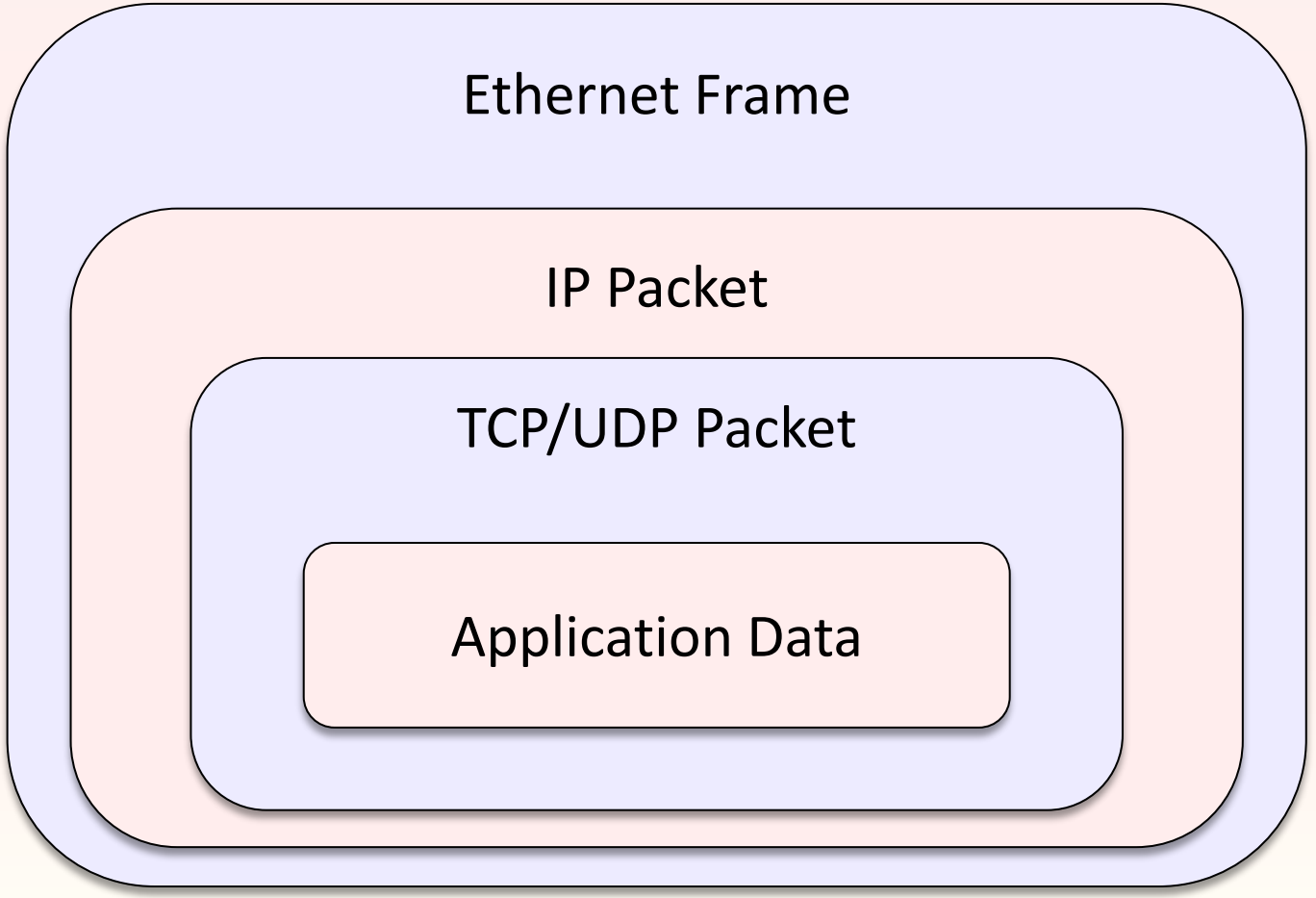
A thin layer above IP

- Sends data to a net socket
- No guarantees.
- No state maintenance or connection setup etc.
- Popular where data can be lost without breaking an application.



TCP/IP: The Simple Picture

You can think of data sent over a network as a nested envelope:



TCP/IP: Software Model

Layer #	Layer Name	Protocol	Protocol Data Unit	Addressing
5	Application	HTTP, SMTP, etc…	Messages	n/a
4	Transport	TCP/UDP	Segments/ Datagrams	Port #s
3	Network or Internet	IP	Packets	IP Address
2	Data Link	Ethernet, Wi-Fi	Frames	MAC Address
1	Physical	10 Base T, 802.11	Bits	n/a

Network protocols are implemented in **software**.

Computer systems implement network protocols as part of their OS (operating system).

TCP/IP: So, what?

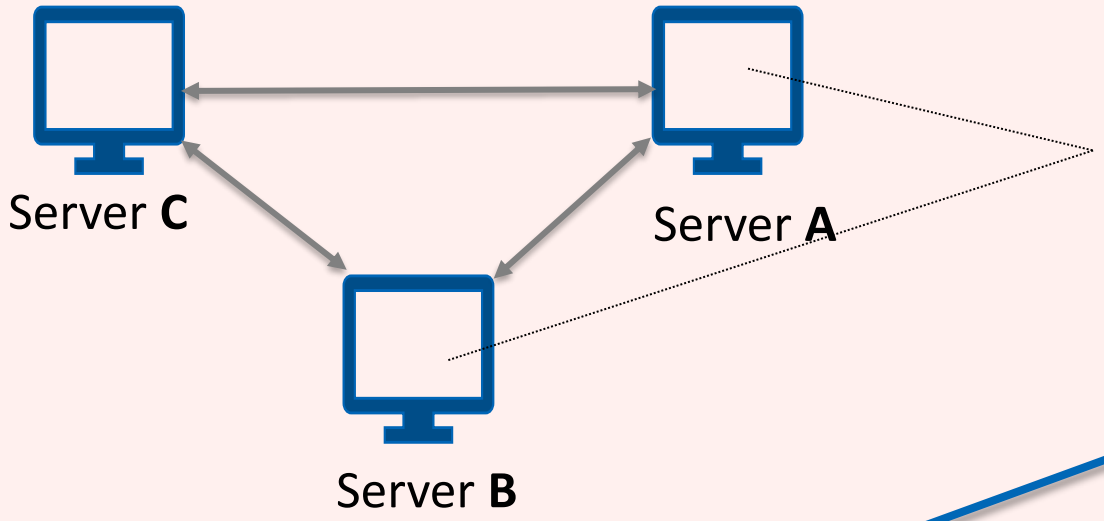
How does this apply to us?

- TCP/IP defines where “things” live in the network stack.
- It also explains how data is sent and how addressing is works.
- It introduces many core concepts and theory that developers of web-based applications need to be aware of.
- It lays the foundation for pretty much all other networking protocols.



Distributed Computing: What is it?

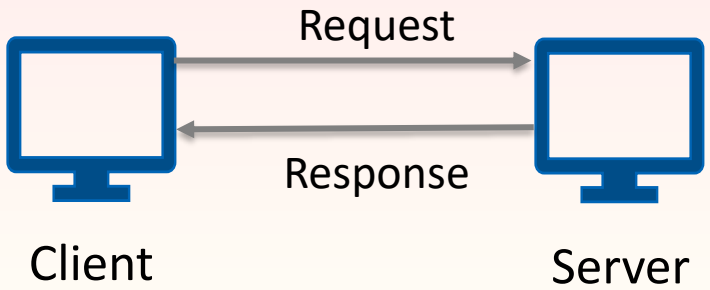
Multiple hardware or software components communicating and coordinate their actions across a network by passing a **message**



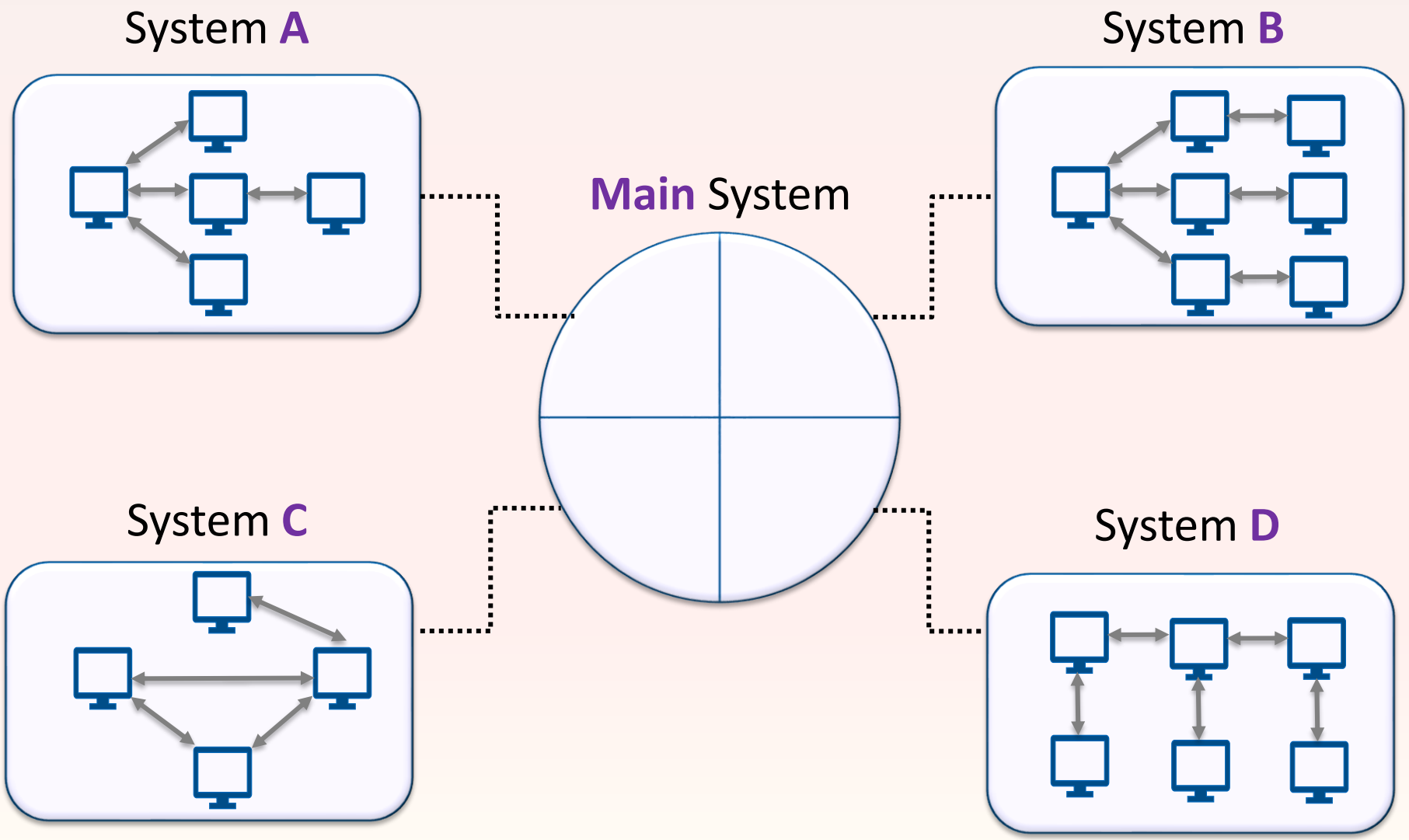
Focuses on
”**resource**” sharing

The most predominant

Client-server *architecture*



Distributed Computing: Functional Separation



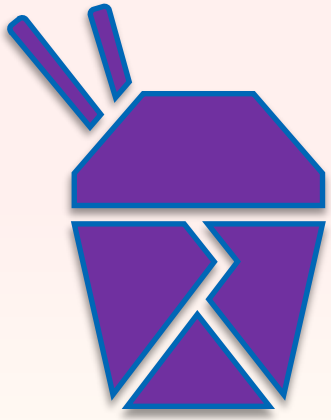
Distributed Computing: The Key Takeaway

Distributed programming:

- Is about coordinated problem solving.
- Constructing and exchanging messages.
- Can get messy without consensus and information.

The key takeaways of this module:

- How and why things get messy.
- How standardisation and design helps prevent it.



Sockets: Not the kind you would stick a fork in

Sockets are a communication endpoint for sending data over a network.

They've been around since the 1970s.

Since then, they have become a fundamental networking primitive.

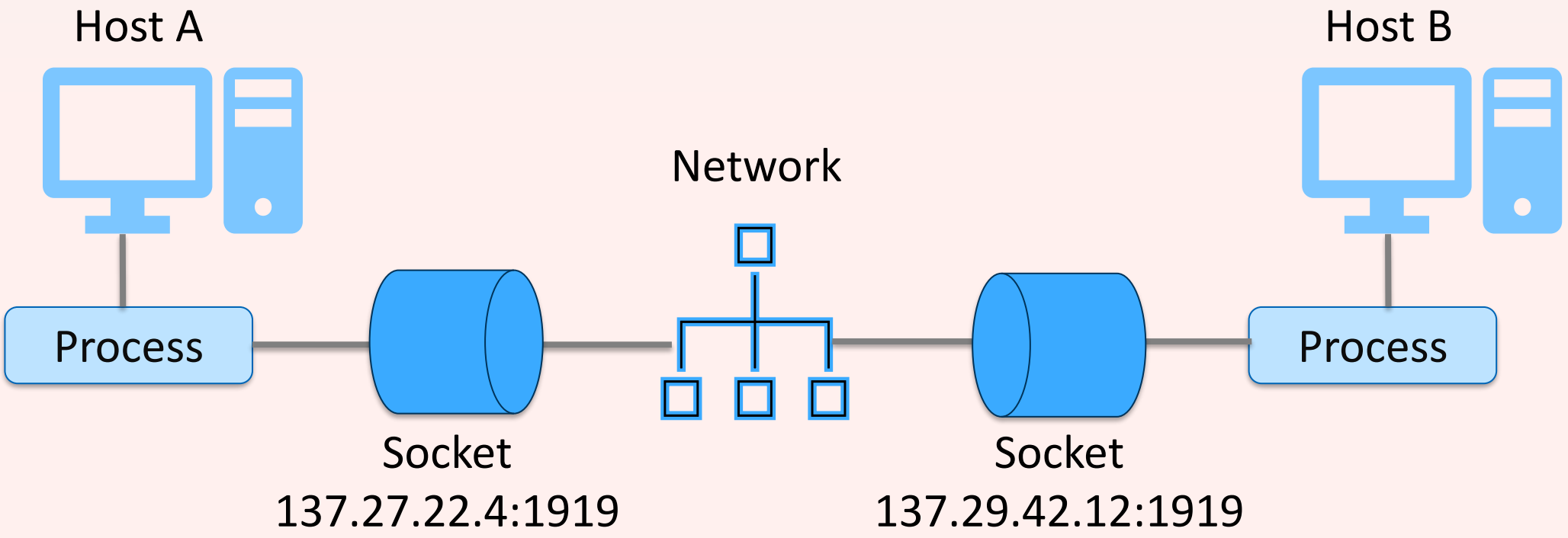
Practically, a socket is an IP address and a port number.

They mainly use TCP or UDP packets, giving us two basic types:

- Connectionless
- Connection-orientated

Ultimately, they act as an abstraction for the many complicated, low-level processes occurring to make data safely pass from one device to another.

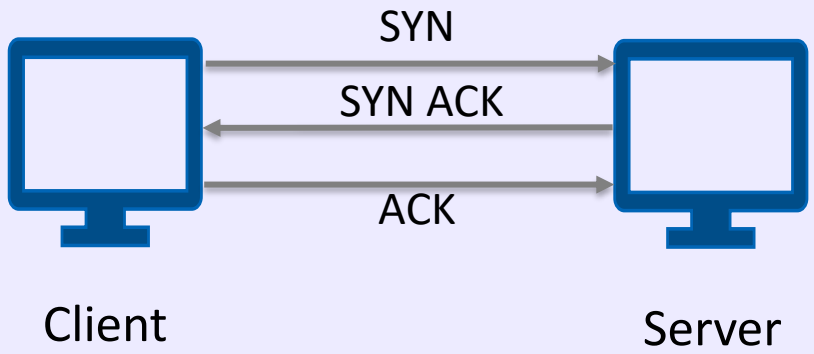
TCP/IP: Sockets



Distributed Computing: Sockets in java.net.*

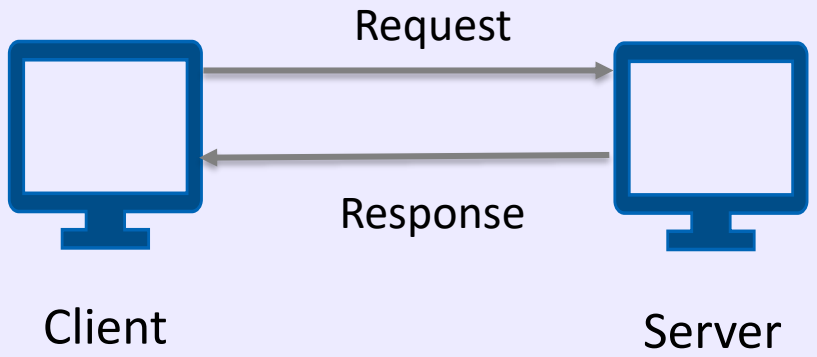
TCP:

- **`ServerSocket`** to listen for client requests.
- **`Socket`** to establishes a virtual connection.



UDP:

- Both hosts communicate via a **`DatagramSocket`**.
- Carry messages using a **`DatagramPacket`**



Socket Programming: Socket and ServerSocket

Client-side:

```
Socket clientSocket = new Socket("10.230.42.12", 4444);
   ...
```

Server-side:

```
SeverSocket serverSocket = new ServerSocket(4444);
Socket in = serverSocket.accept();
   ...
```

Socket Programming: DatagramPacket and DatagramSocket

Client-side:

```
byte[] buf = new byte[256];

DatagramSocket socket = new DatagramSocket();

DatagramPacket packet = new DatagramPacket(buf, buf.length,
                                           ipAdd, 2929);

socket.send(packet)

. . .

buf = new byte[256];

packet = new DatagramPacket(buf, buf.length);

socket.receive(packet);
```

Socket Programming: DatagramPacket and DatagramSocket

On the server side:

```
DatagramSocket socket = new DatagramSocket(4444);
 // Wait for client request

byte[] buf = new byte[256];
DatagramPacket packet = new DatagramPacket(buf, buf.length);
socket.receive(packet)

// Make a new buffer of data to send back
packet = new DatagramPacket(buf, buf.length, address, port);
socket.send(packet);
```

Concept Check: Data as a Sequence

Anyone notice the key difference between TCP and UDP sockets?



Socket Programming: TCP vs UDP

UDP:

- Connectionless
- Packet-orientated

That means data will be sent and received as an isolated chunk of bytes.

There is no expected order or sequence, only an allocated size.

TCP:

- Connection-orientated
- Sends a sliding windows of packets

So how does that change the programming?

Socket Programming: I/O Streams

Streams! Sequences of raw bytes make a stream of primitive data.

That, in turn, adds up to more complex data.

You've actually been using streams since your first year.

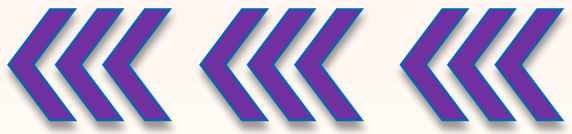
They've just been hidden under syntax:

`System.in`

`System.out`

`in` and `out` are just public attributes of that class.
Their data type?
`InputStream` and `PrintStream`

`System` is just a utility class that talks to your operating system.



Socket Programming: I/O Streams – Cont.

There are *many* different types of streams for different use cases.

To name a few:

AudioInputStream, ObjectInputStream, FileInputStream

PipedOutputStream, BytesArrayOutputStream, FilterOutputStream, ….

TCP Sockets allow us to send and receive data over a network.

Want to guess how that data arrives?

Yes, as I/O streams!



Socket Programming: I/O Stream Wrappers

Raw byte streams are great, but …

Working with the (input or output) can require a lot of manual processing.

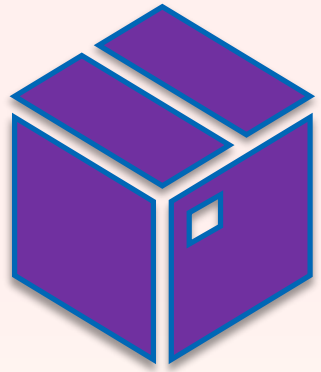
This is where higher-level objects, known as wrappers become useful.

Up to this point, you've likely used the Scanner class for a similar I/O management in Java.

For example:

```
Scanner consoleIn = new Scanner(System.in);

int number = consoleIn.nextInt();
```



The Scanner is just one of the simplest ways to manage I/O streams.

Socket Programming: I/O Stream Readers and Writers - Client

Client Side:

```
Socket clientSocket = new Socket("127.0.0.1", 3939);
InputStream in = clientSocket.getInputStream();
OutputStream out = clientSocket.getOutputStream();
BufferedReader br = new BufferedReader(
                                    new InputStreamReader(in)
                                    );
PrintWriter pr = new PrintWriter(out);
pr.println("Sending this message to server");
pr.flush();
```

Socket Programming: I/O Stream Readers and Writers - Server

Server Side:

```
ServerSocket serverSocket = new ServerSocket(3939);
Socket clientSocket = serverSocket.accept();
InputStream in = clientSocket.getInputStream();
OutputStream out = clientSocket.getOutputStream();
BufferedReader br = new BufferedReader(
                                    new InputStreamReader(in)
                                    );
PrintWriter pr = new PrintWriter(out);
pr.println("Sending this back to the client");
```

DEMO

TCP and UDP Networking Applications using Java Sockets

Distributed Computing: Better Models

Socket based networking allows basic distributed exchange.

Not ideal for calling remote API's.

Additional code is needed to deal with versioning, protocol changes, marshalling and unmarshalling (converting objects to transferable data)

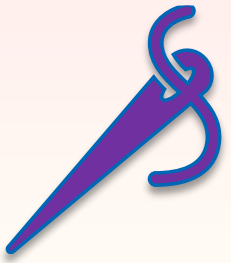
Help comes in the form of:

- Java RMI
- MSDCOM (Microsoft Distributed Component Object Model)
- CORBA (Common Object Request Broker Architecture).

While we will only briefly look at Java RMI, the others introduce some interesting ideas regarding platform independent communication between software.

WEEK 2

CPUs, Threads and Concurrency



CPUs and Threads: The Thread

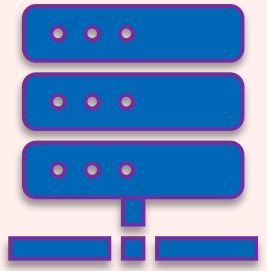
- As far as we, and the OS, are concerned, these workers are implemented as threads.
- Threads are the smallest unit of execution that can be scheduled by an OS or runtime environment.
- Processes are "larger" collections of work and can be made up of different threads.
- The programs we have been writing are single-threaded.
- That meant any 'thread blocking' methods we called from the server socket 'stopped' our server program until a request came from the client.
- What does that mean for us?
- Basically, we can deal with more than one client request at a time.

Concurrency: Client Requests

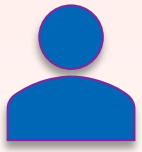
Think back to the simple server sockets from last week.

They were single-threaded and could only deal with one request at a time.

Server



Waits for connection



Client A

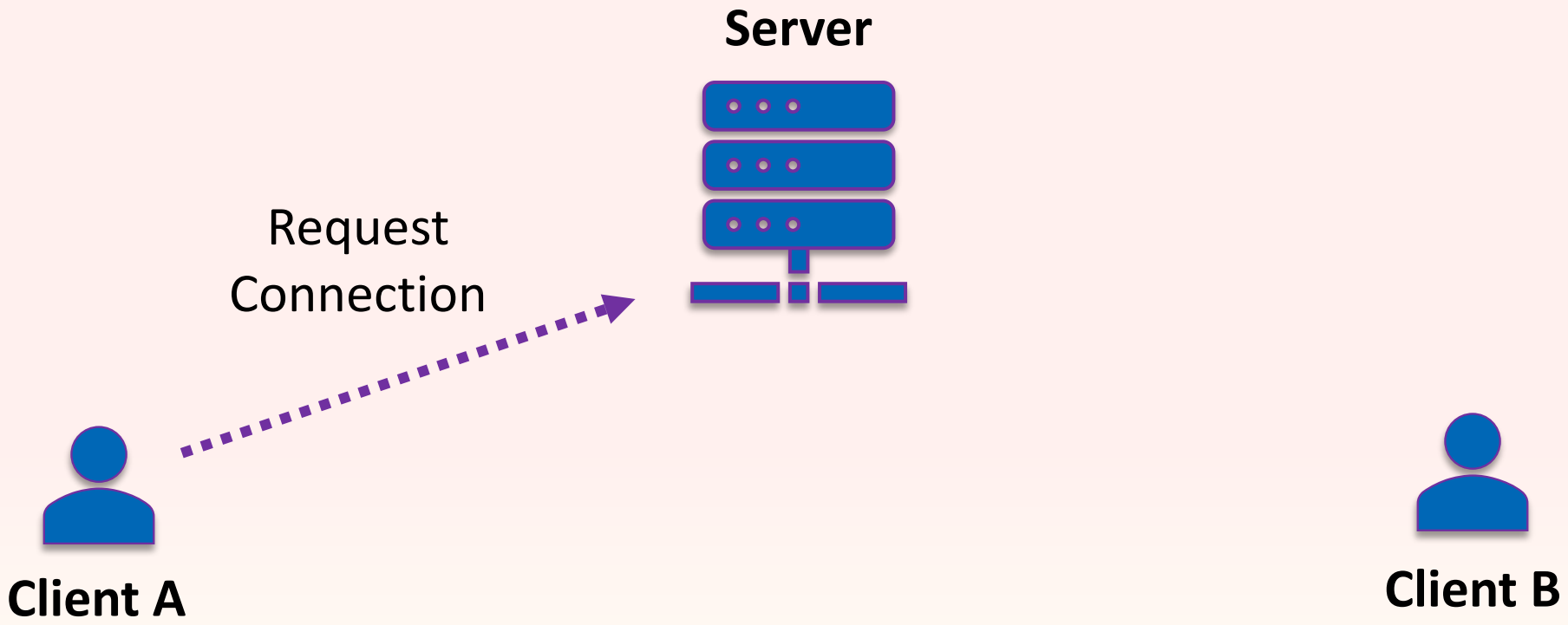


Client B

Concurrency: Client Requests

Think back to the simple server sockets from last week.

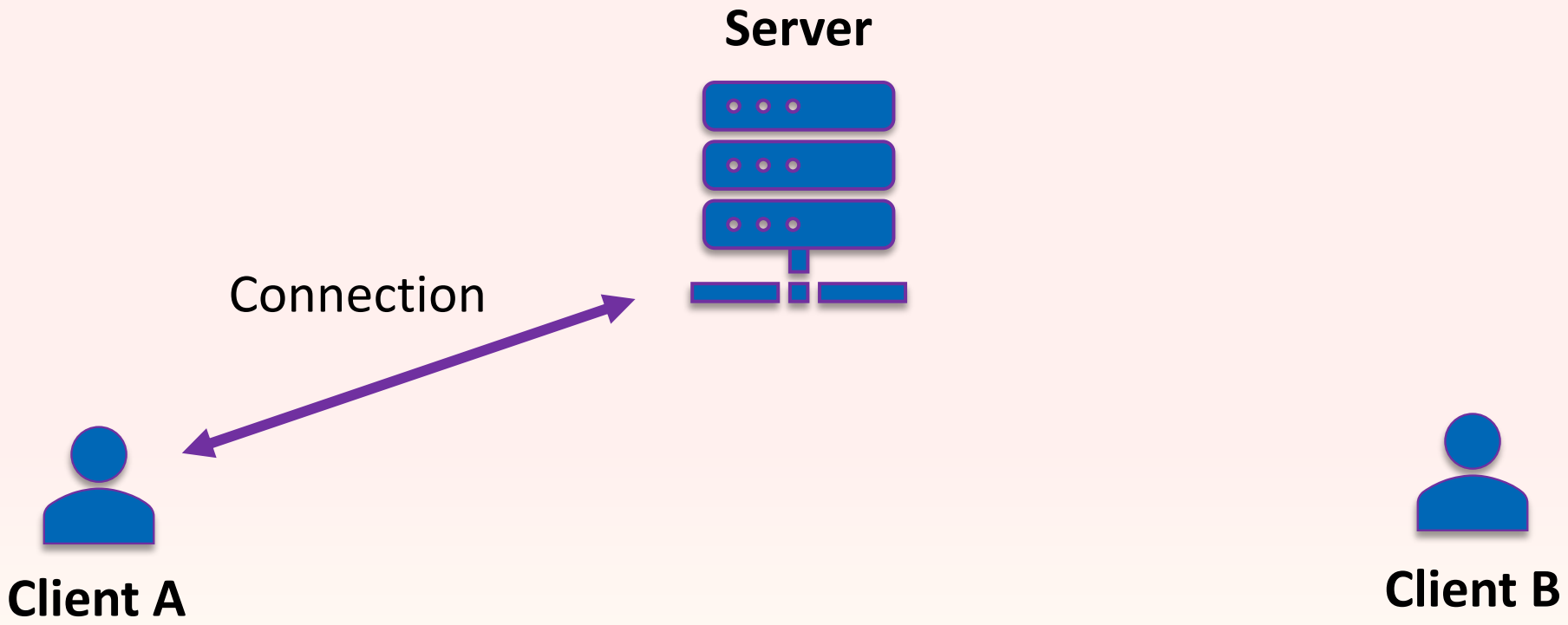
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

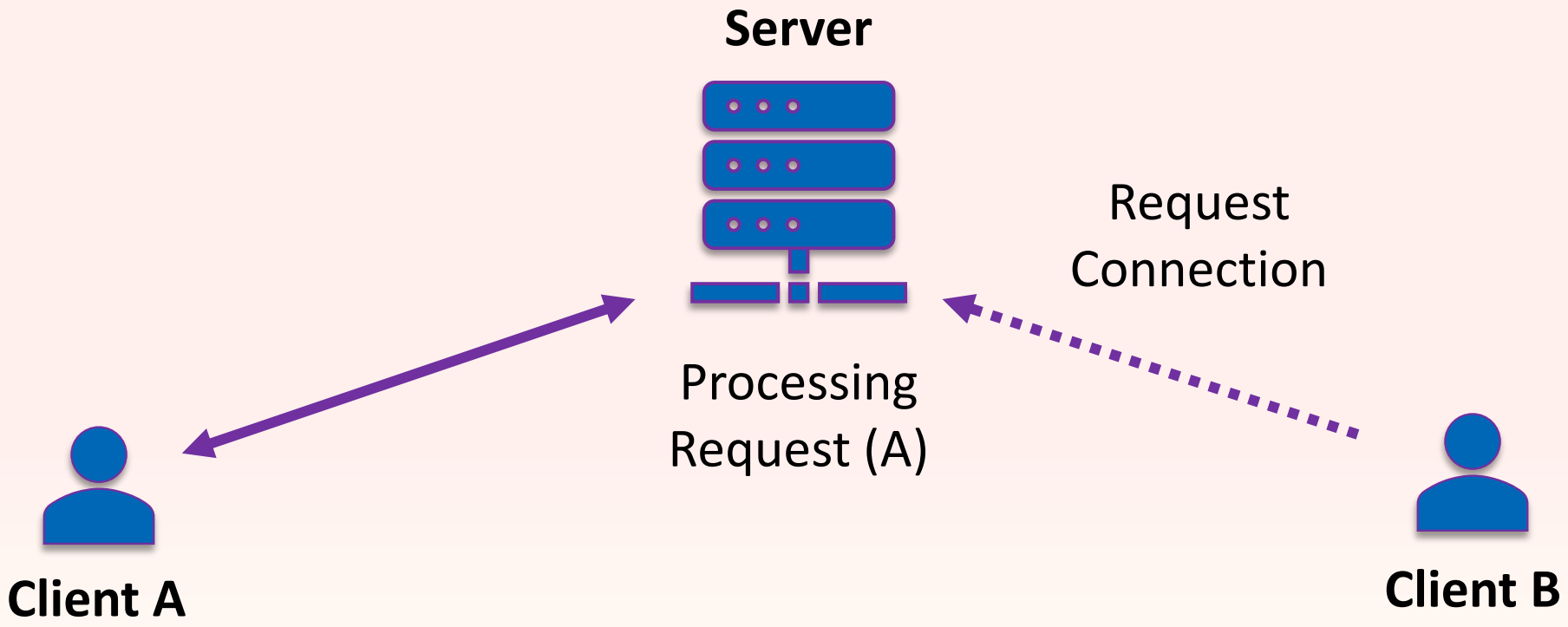
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

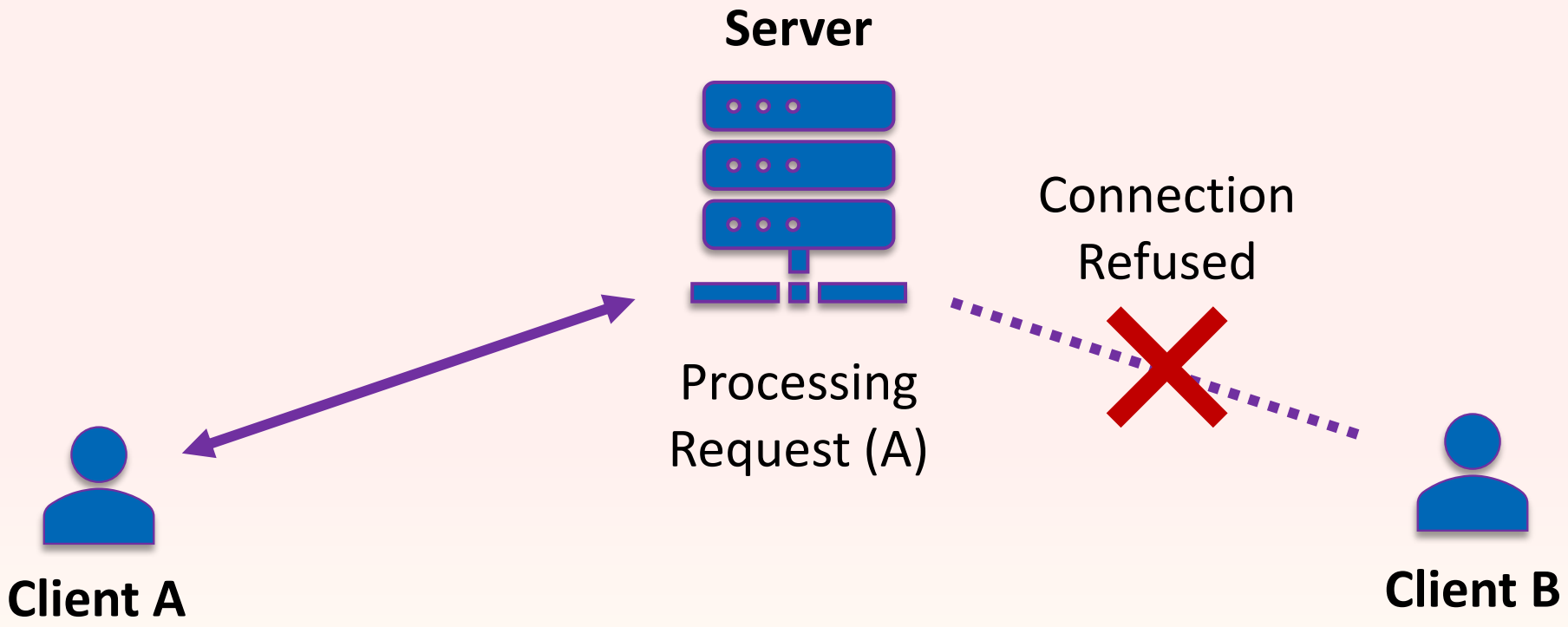
They were single-threaded and could only deal with one request at a time.



Concurrency: Client Requests

Think back to the simple server sockets from last week.

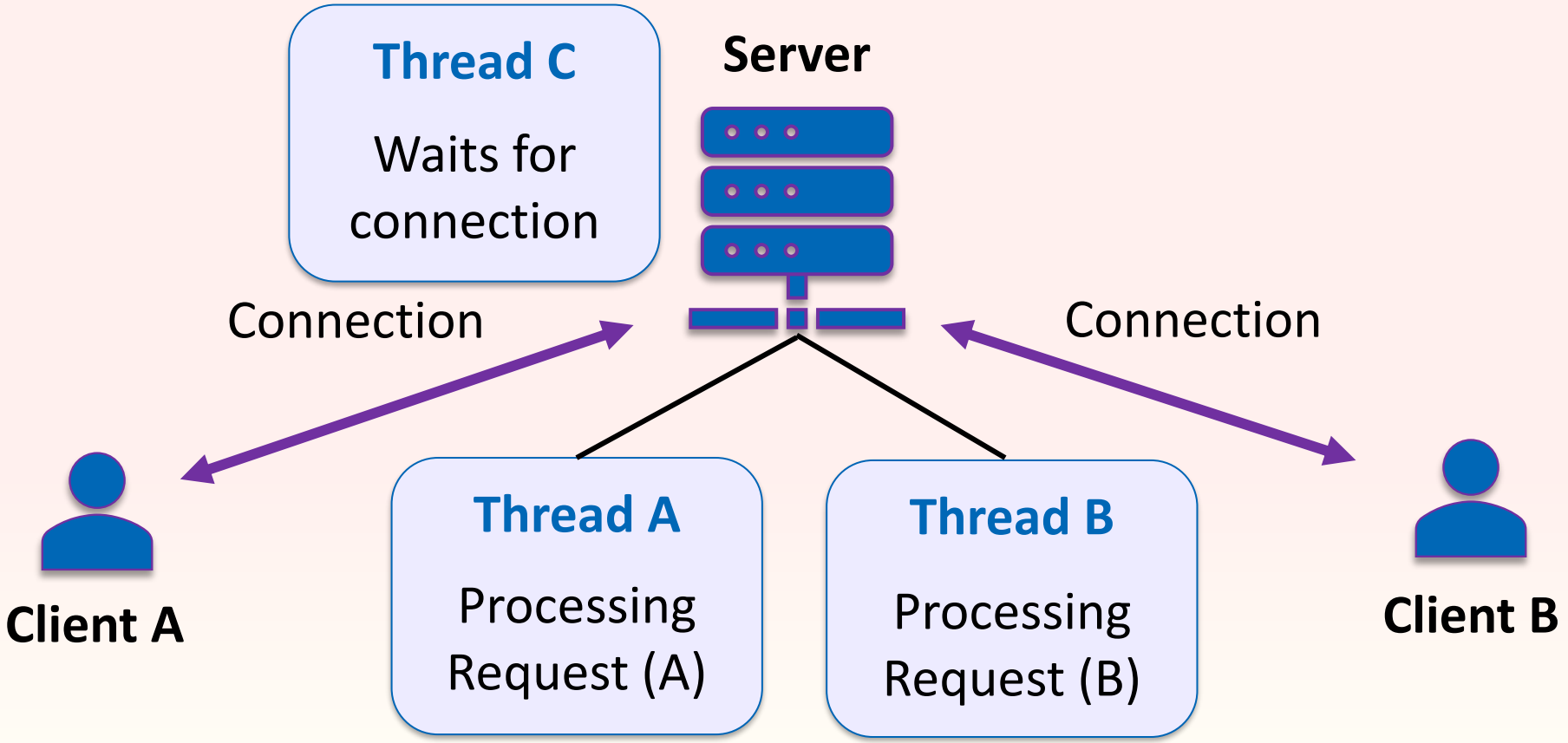
They were single-threaded and could only deal with one request at a time.



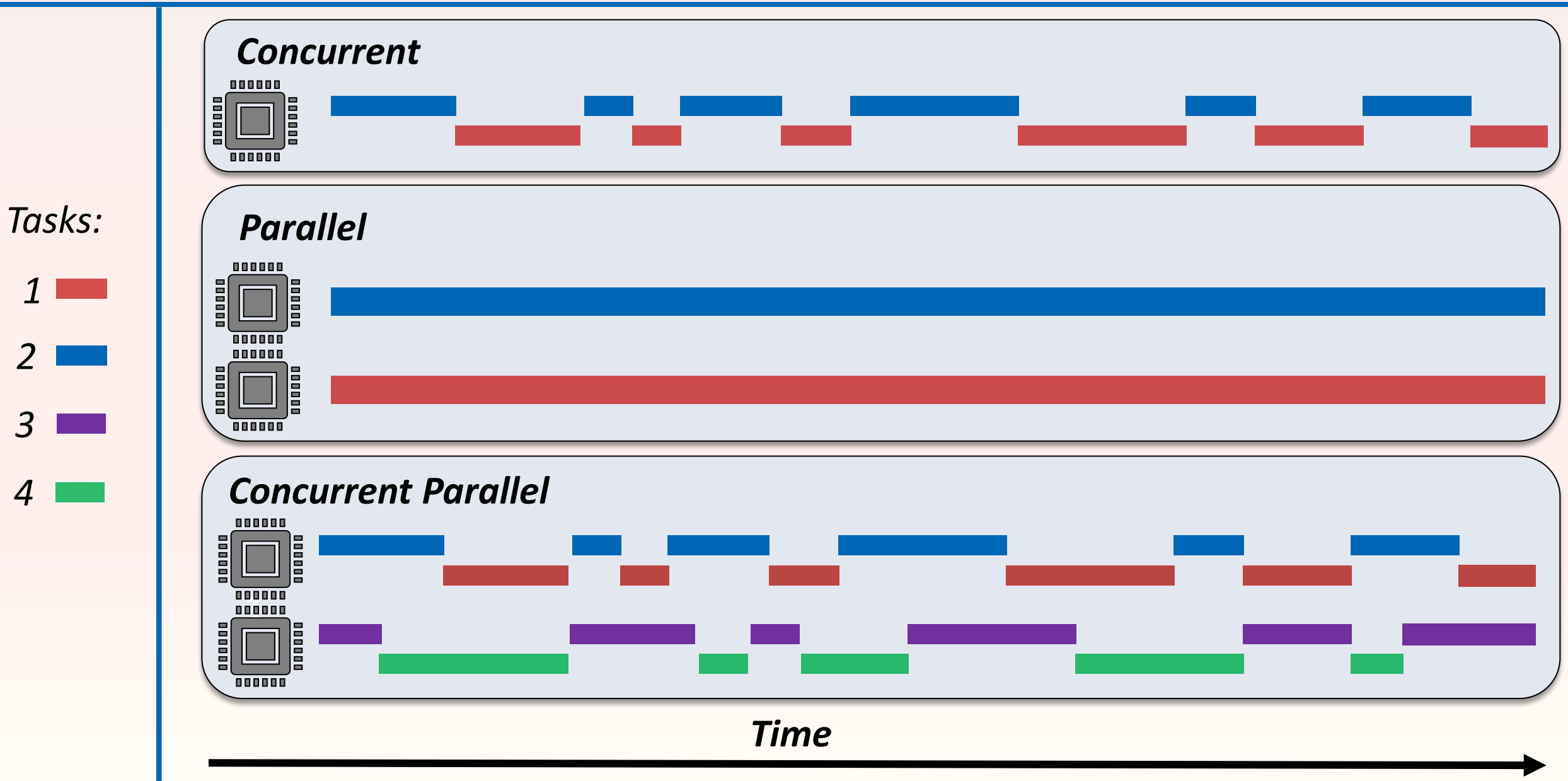
Concurrency: Client Requests - Multithreaded

If the server can spawn new threads as and when it needs them, it can both process existing requests *and* listen for new connections.

The number of clients and requests scales with the CPU cores of the server.



Concurrency: Not Parallelism



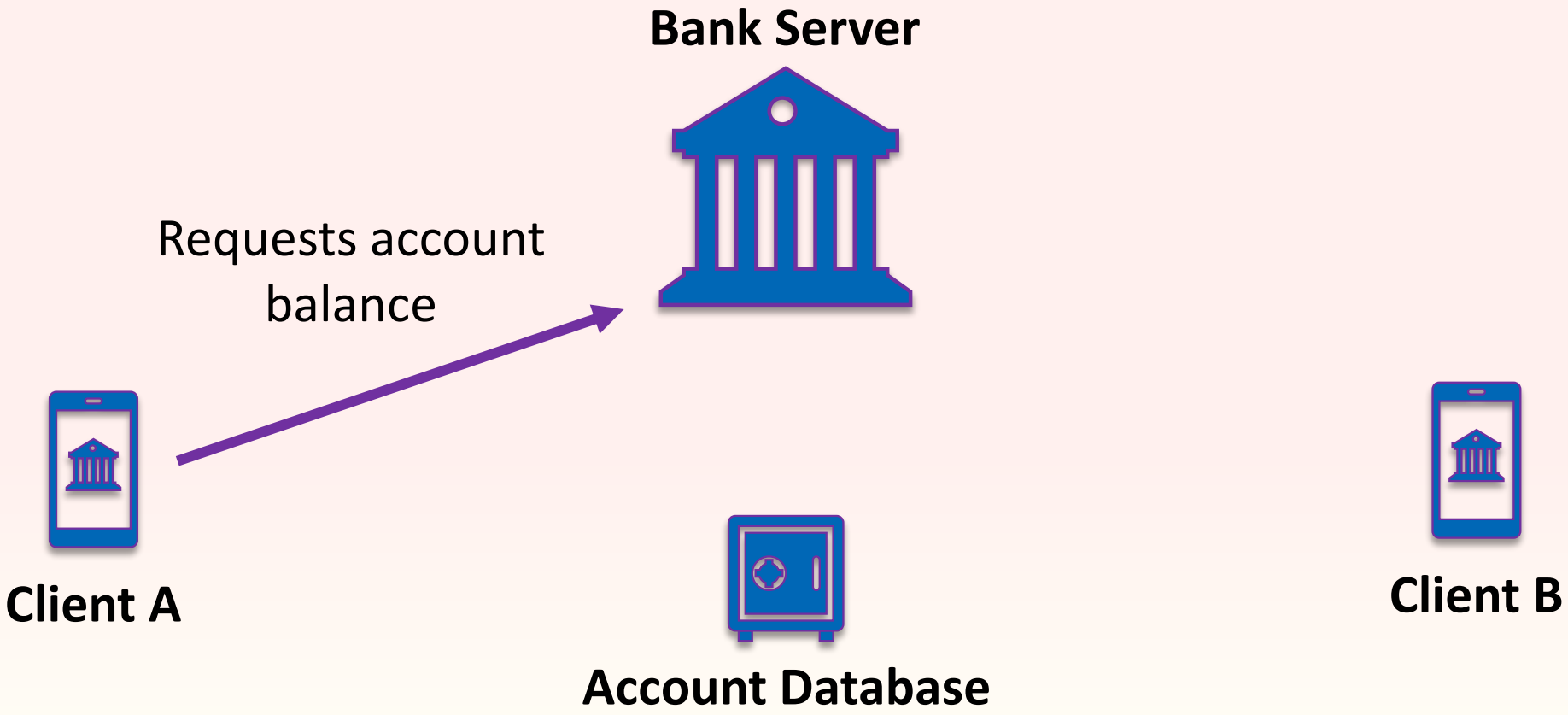
CPUs and Threads: Thread Lifecycle

State	Description
New	Thread had been created and can be started.
Runnable	Thread is performing a task. The thread is 'running' when assigned window of CPU time. Once that time window expires, it has the status 'runnable' and is waiting for more CPU time.
Blocked	Thread is waiting on some resource to become available, or another task to finish before it can continue.
Waiting	Thread has been asked to, or chosen, to wait because another task must be completed before it resumes. It resumes once it's notified to.
Timed Waiting	Thread is asked to (or chosen to) wait for a set number of time units, or until notified to continue.
Dead	Thread has terminated, finishing its task or experiencing an error.

Concurrency: Readers and Writers

Consider the following:

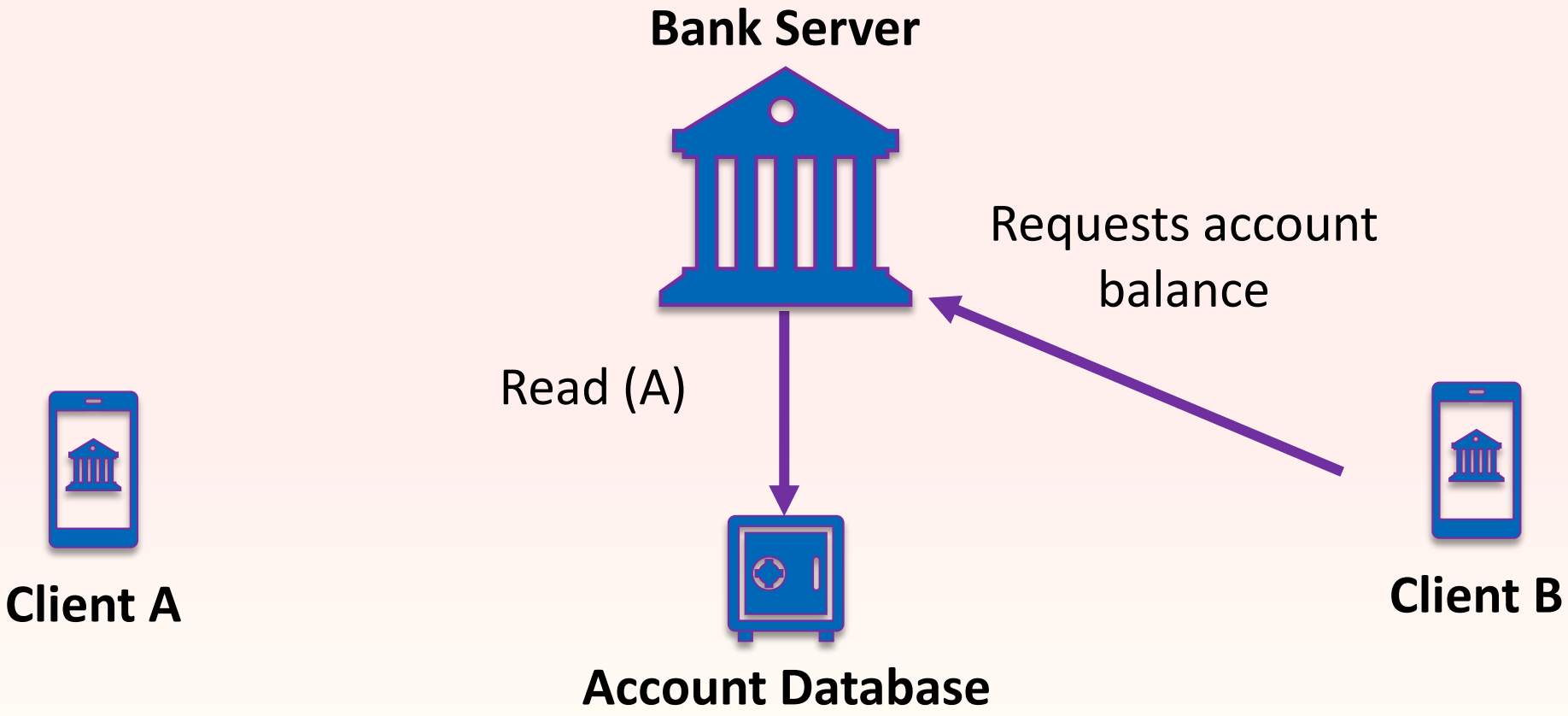
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

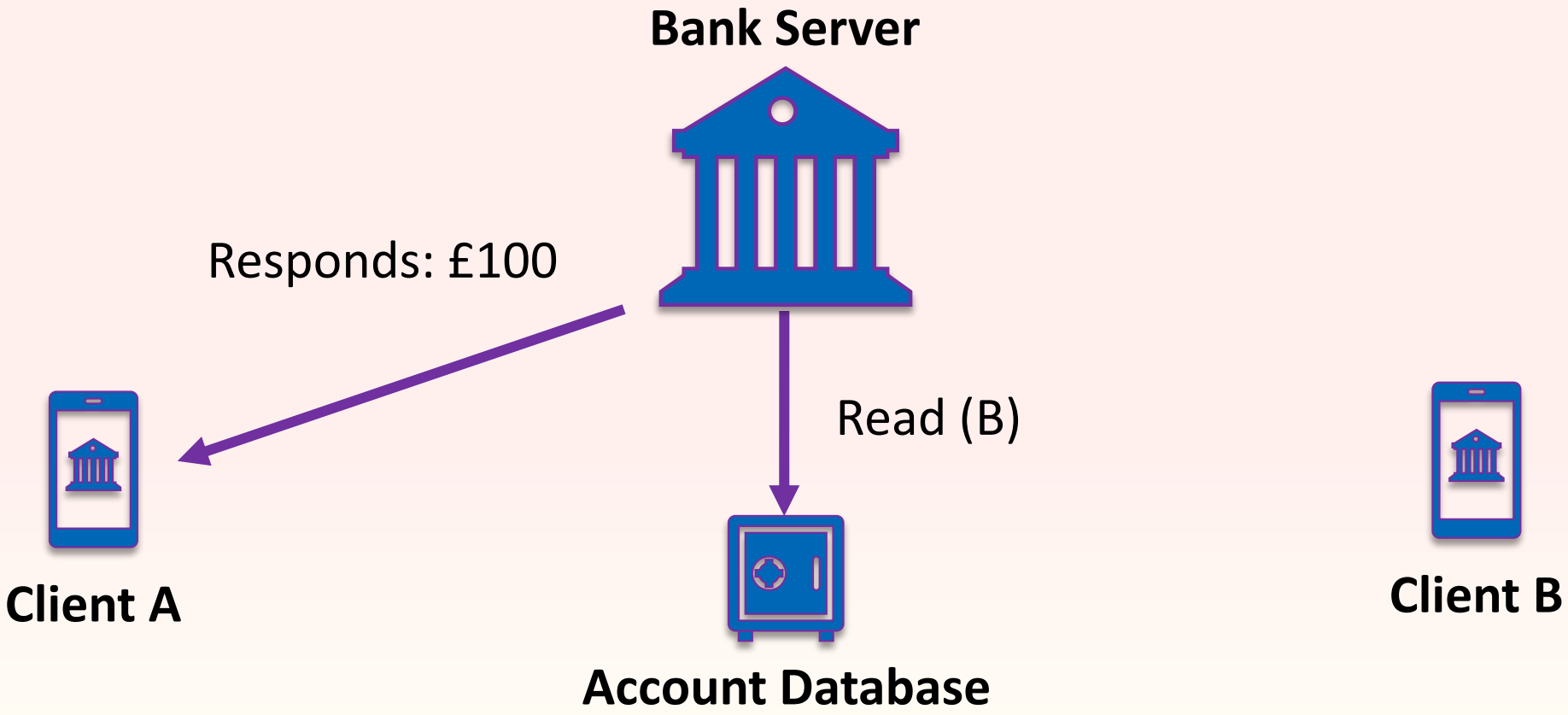
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

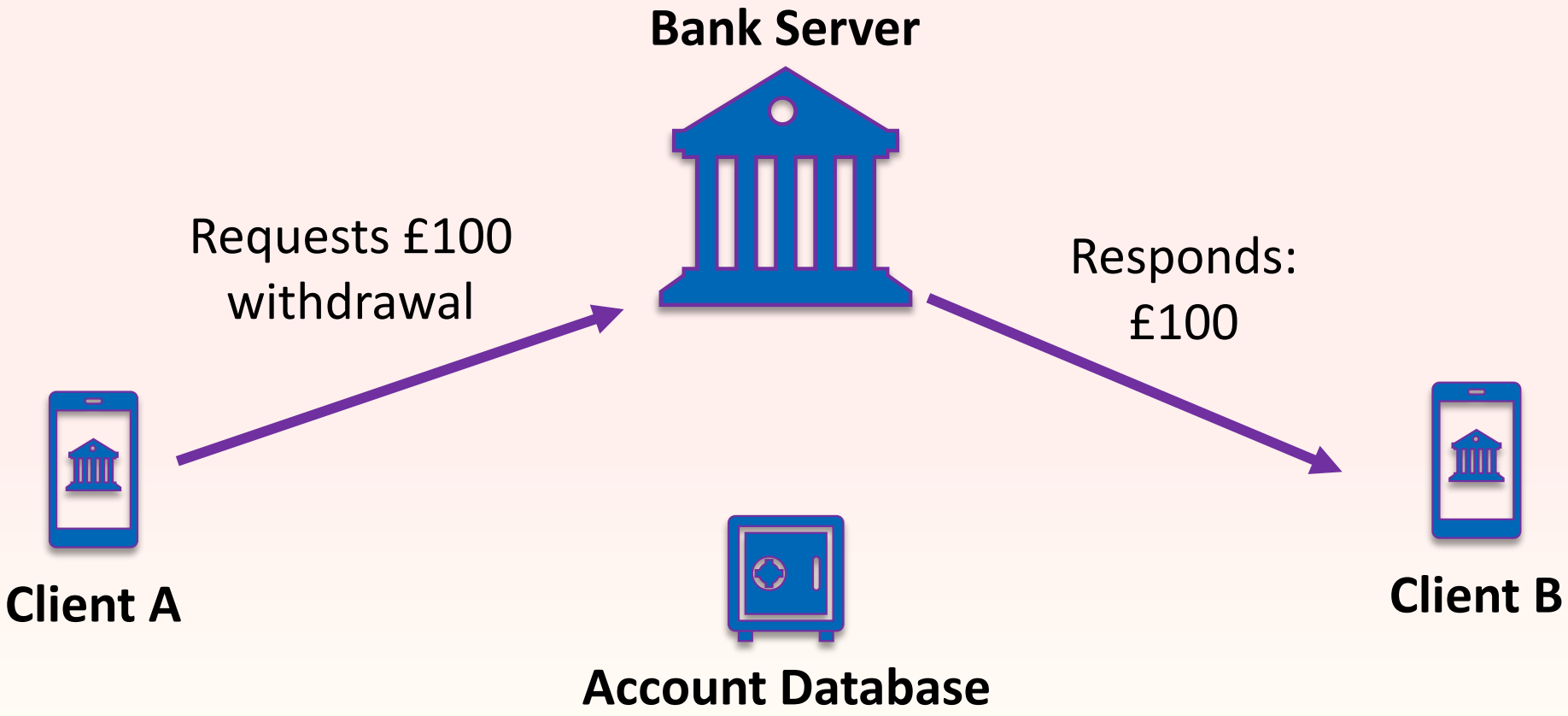
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

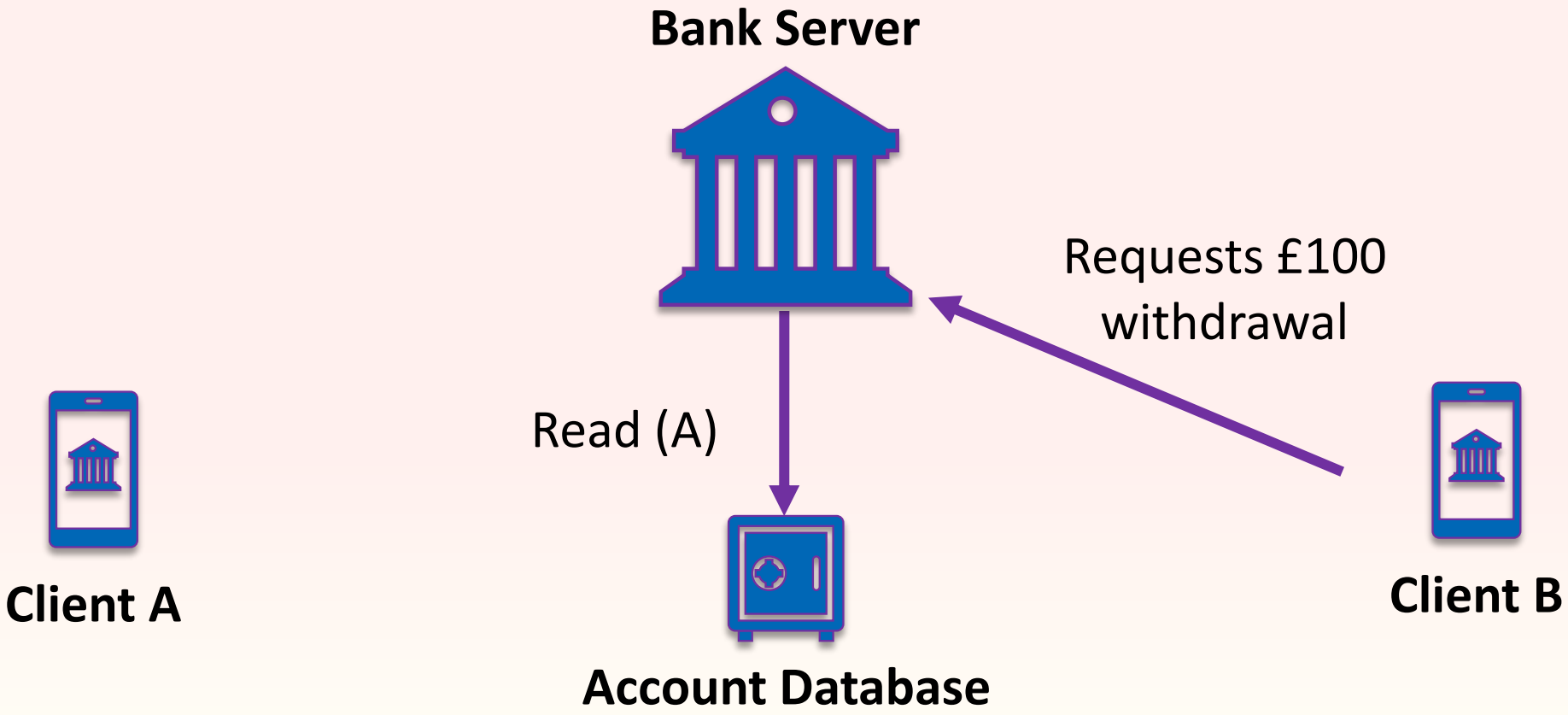
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

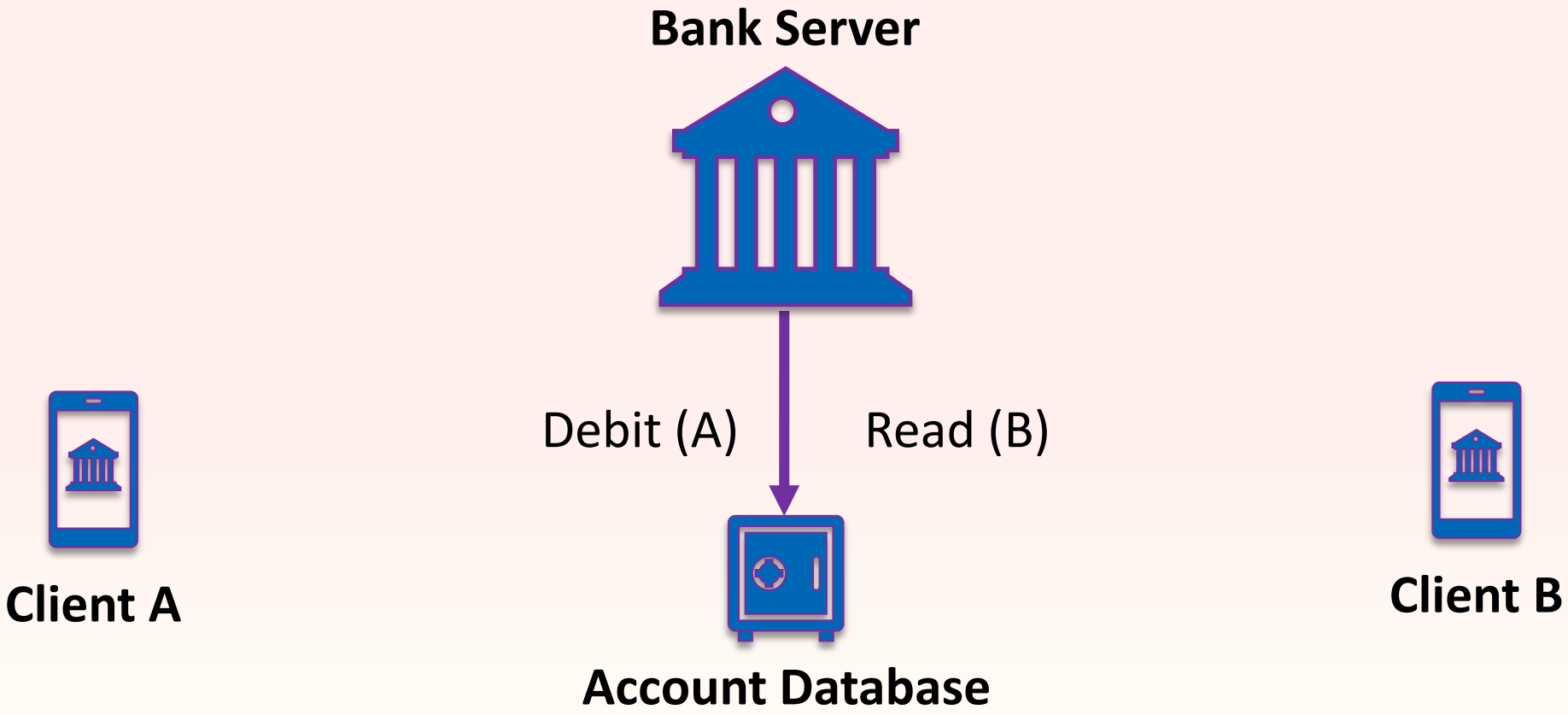
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

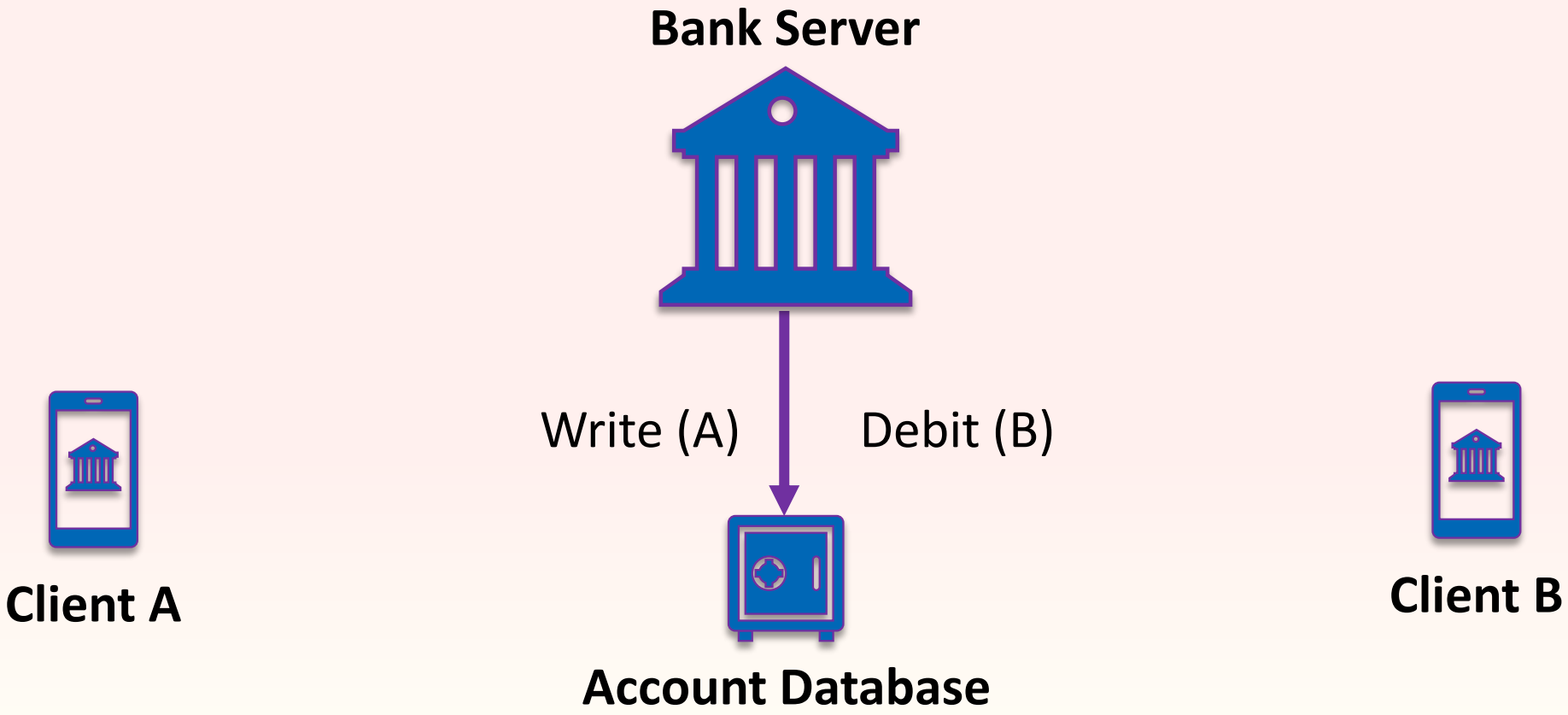
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

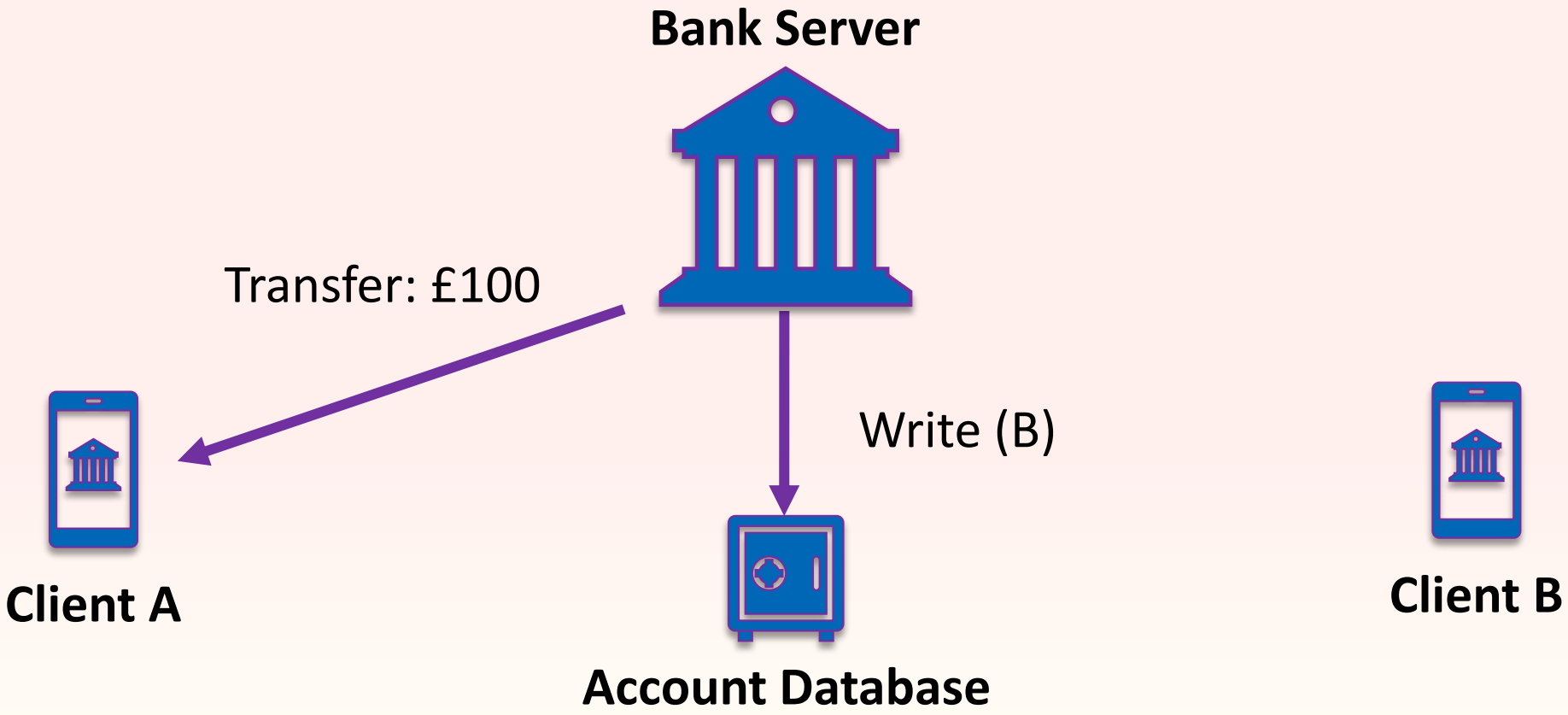
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

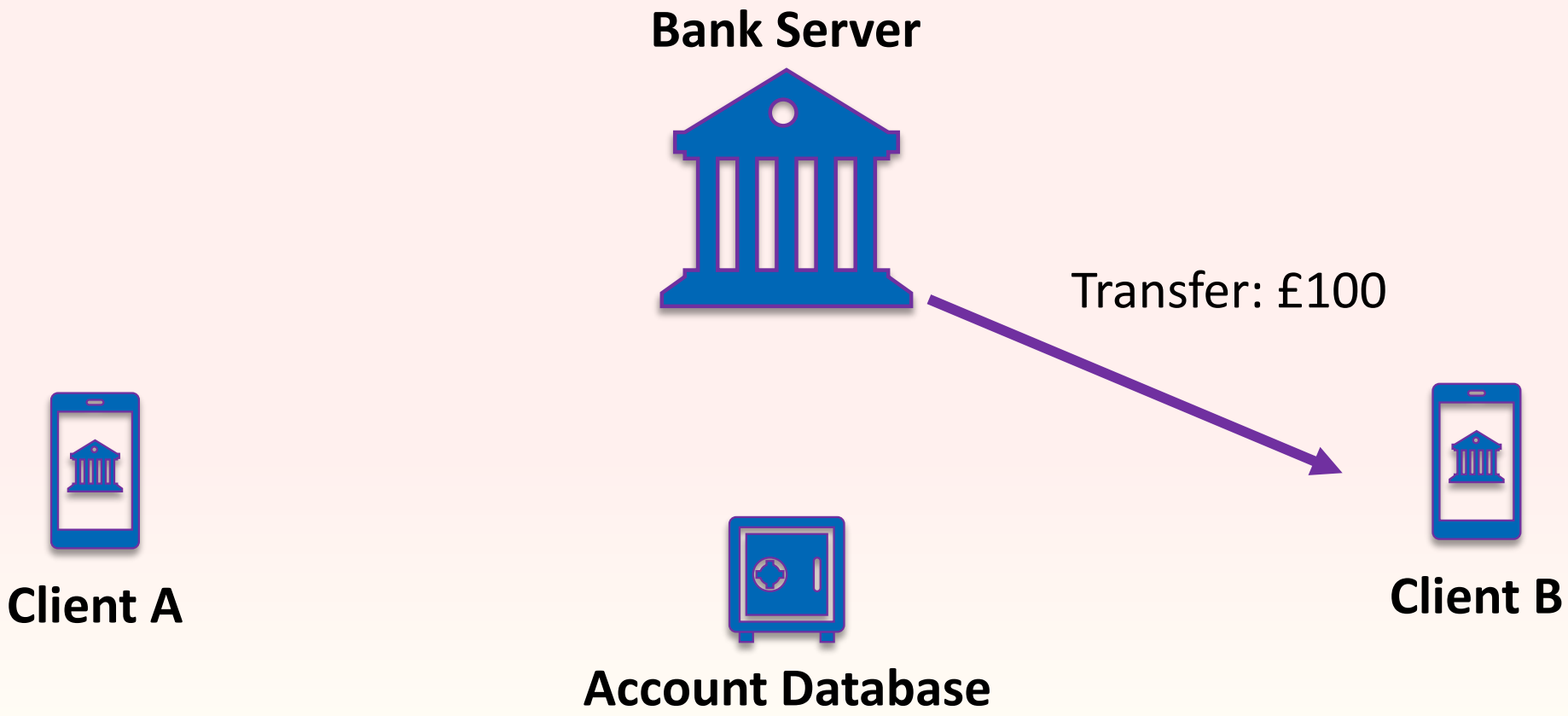
We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?



Concurrency: Readers and Writers

Consider the following:

We have a distributed banking service that stores shared data and allows clients to interact with it. What problems might occur?

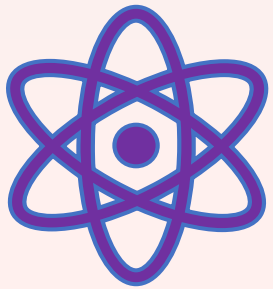


Concurrency: Key Takeaways

We need to pay attention to where operations:

- Read data
- Write data

And understand the difference between them.



Concurrent writes cannot be allowed.

- We can only allow one at a time.

Concurrent reads can happen, but:

- They cannot happen during a write.

Critical operations, when started, should not be interrupted!

In other words, they should be atomic

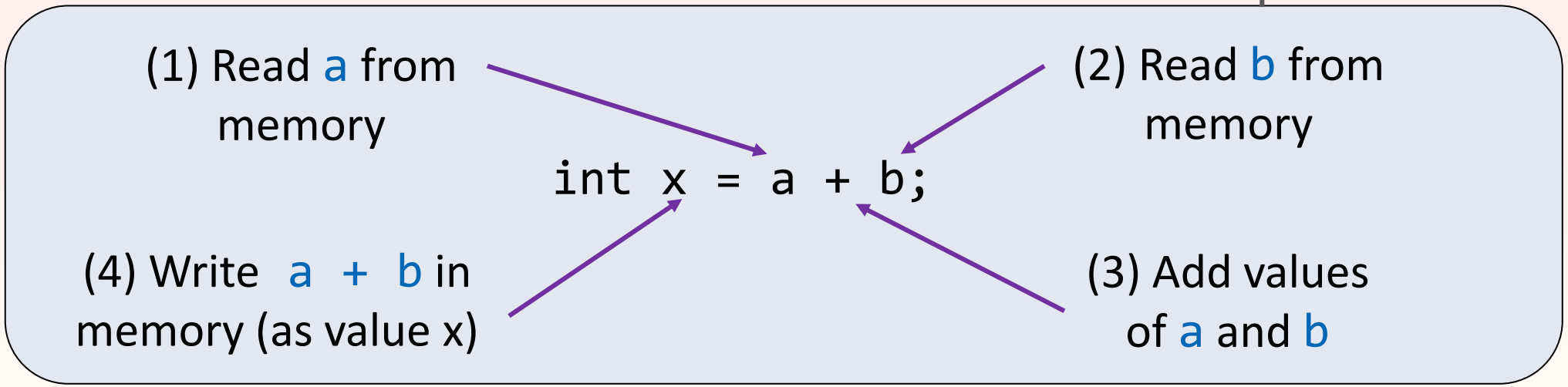
Concurrency: Atomics and Programming

Consider this Java snippet. Specifically, the third statement.

```
int a = 10;
int b = 5;
int x = a + b;
```

A thread managing this task can be interrupted by another thread running within the same process!

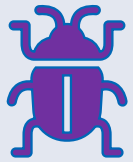
How many operations are happening here?



Concurrency: What could go wrong?

Application Errors

- Software doesn't do what you want it to do
- Programs execute in an 'unexpected' manner.
- Ultimately, the 'solution' is wrong.



Deadlock

- Multiple workers hold a resource needed by others.
- No one will release the resource until the other becomes available.
- Ultimately, everything stops.



Lockout

- Workers access resources to complete tasks with no queuing.
- Those workers indefinitely prevent another worker from accessing a resource it needs.
- It gets 'locked out'



Concurrency: So, what now?

Concurrency can lead to some horrific errors when things go wrong.

Those errors can be extremely difficult to debug and reproduce.

You cannot predict when a thread will finish one operations and move on to the next.

Fortunately, this isn't a new problem.

- Most programming languages have native support for writing concurrent software and controlling threads.
- But as you can imagine, the onus is on the programmer to avoid shooting themselves (and others) in the foot.

Threads in Java: Classes and Interfaces

There are two basic ways of managing Thread objects in Java.

Extending the Thread class.

- Overrides a method called `run()` which houses the code your threads will execute.
- Generally simpler, but less flexible and not very “Object Orientated”.
- Java’s single inheritance means you can’t extend any other class.

Implementing the Runnable interface.

- Forces your class to implement a `run()` method, but doesn’t necessarily violate the stylistic purpose of inheritance
- Any objects implementing runnable can then be used as a Thread object parameter, meaning the task can be passed between threads.

Threads in Java: Extending Thread - Run

```
class MyThread extends Thread {
@Override
public void run() {
 for (int i = 1; i <= 5; i++) {
   System.out.println(this.getName() + " - Task: " + i);
   try {
       // Do some work
   } catch (InterruptedException e) {e.printStackTrace(); }
   }
 }
 }
```

Threads in Java: Extending Thread - Main

```
public class Main {
 public static void main(String[] args) {
   MyThread th1 = new MyThread();
   MyThread th2 = new MyThread();
   th1.start();
   th2.start();
   }
 }
```

Threads in Java: Implementing Runnable - Run

```
class MyTask implements Runnable {
@Override
public void run() {
   for (int i = 1; i <= 5; i++) {
      try {
            // Do some work
      } catch (InterruptedException e) {
            e.printStackTrace();
      }
      }
   }
```

Threads in Java: Implementing Runnable - Main

```
public class Main {
 public static void main(String[] args) {
   MyTask task = new MyTask();
   Thread th1 = new Thread(task);
   Thread th2 = new Thread(task);
   th1.start();
   th2.start();
   }
}
```

Note:
Our class implementing Runnable is now passed to the Thread class constructor.

Multithreading: Monitors and Locks

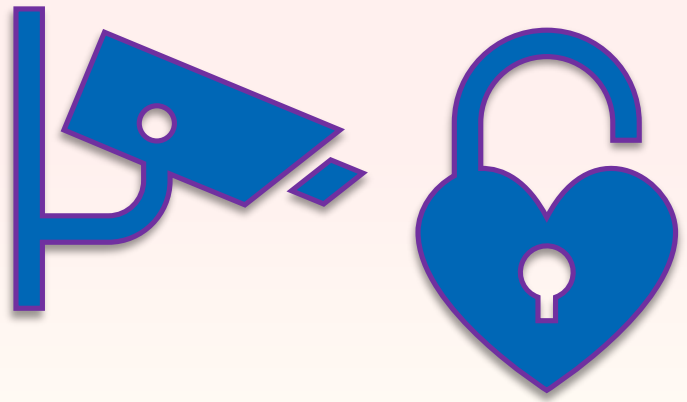
Okay, so we can cause race conditions in Java. Yay!

As mentioned last lecture, many languages introduce mechanisms to help prevent them.

In Java there are quite a few ways of controlling threads.

We're going to look at two lower-level approaches. These are:

- Locks
- Monitors



Multithreading: Monitors and Synchronisation

Generally, monitors are somewhat of an abstract notion.

Naturally, that means there are different implementations of them.

As far as we're concerned, they:

- Ensure only one thread can execute a specific block of code at a time.
- Use condition variables to control certain stages of a thread's life cycle.

Java supports monitors via the:

- `synchronized` keyword
- `wait()`, `notify()` and `notifyAll()` methods

Multithreading: synchronized Methods

```
public class synchronized Counter {

      private int c = 0;
      public synchronized void inc() {
            c++;
      }
      public synchronized void dec() {
            c--;
      }
      public synchronized int value() {
            return c;
      }
}
```

The synchronized keyword can be used in method signatures.

If multiple threads can access this object in shared memory:

- Only one thread can call a synchronized method at a time.
- All other threads must wait for the 'current' thread to finish before they can execute any other synchronized code.
- In other words, they are mutually exclusive (mutex)

Multithreading: `synchronized` Methods

Synchronised methods are useful, but they:

Can be inefficient

- The entire method (and all other synchronsized) is locked.
- The ‘critical’ operations might only be a small part of the method

Only ensure mutual exclusion for a *shared* object, not for arbitrary data!

- We may have an `ArrayList` we want our thread objects to interact with
- Synchronising access to the list would mean making a new wrapper class to control data manipulation through synchronised methods.
- That can be both inefficient and awkward.

Multithreading: synchronized Blocks

```
List<Socket> clients = new
                    ArrayList<>();

synchronized (clients) {
        clients.add(clientSocket)
}

synchronized (clients) {
        clients.remove(clientSocket)
}

synchronized (clients) {
       System.out.println(
             "Connected: " + clients)
}
```

The `synchronized` keyword can be used to create blocks.

- The block is parameterized by an object.
- Any other `synchronized` block, also parameterised by the *same* object, becomes mutually exclusive.

Note: Using `clients` as your block parameter doesn't 'protect' it. This just serves as a 'common' object to tie collections of synch blocks mutually exclusive.

Multithreading: synchronized Blocks – Common Objects

```
Object lock = new Object();

List<Socket> clients = new
                    ArrayList<>();


synchronized (lock) {
      clients.add(clientSocket)
}

synchronized (lock) {
      clients.remove(clientSocket)
}
```

We can just as easily:

- Create a new object and call it `lock`
- Use it exclusively, as our `synchronized` parameter.

Once inside the synchronized block:

- perform the critical operations on the object of interest.

`lock` will then be our reference for synchronising operations on the list of clients

Multithreading: `synchronized` Blocks - Gotchas

```
List<Socket> list1 = new ArrayList<>();
List<Socket> list2 = new ArrayList<>();

synchronized (list1) {
      if (list2.size() > 1000) {
            list1.add(clientSocket)
      }
}

synchronized (list2) {
    if (list1.isEmpty()) {
            list2.remove(clientSocket)
      }
}
```

Consider this example:

- We have two lists whose operations rely on each other.

What might be the problem?

- Their synchronsiation blocks do not share a common object!
- Operations on `list2` depend on the contents of `list1`
- But their blocks are not mutually exclusive!

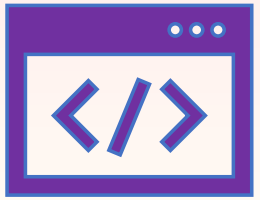
Multithreading: Summarising Readers and Writer

Read operations need to increment and decrement counters using synchronised methods because:

- Write operations might already be happening.
- The counter will need to wait for the write to finish. Once that happens, it can increment the counter.
- Once the counter change starts, no write operations can go ahead and will have to wait.
- Counters decrement when finishing a read, preventing write operations until notify is called and briefly preventing other read operations from changing the counter.
- If a write operation starts, then it can only enter the synchronized block if no other threads are also trying to write or there are ‘active’ reads.

WEEK 3

Object Oriented Distributed Programming and Web Technologies



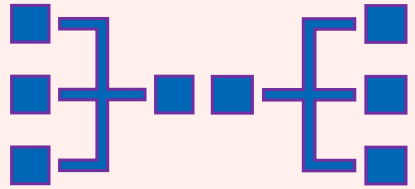
Object Oriented Approaches: Abstracting Sockets

Object Oriented paradigms gain traction in the late 1980s and 1990s.

Computer systems are becoming complex.

Distributed programming is becoming desirable.

Developers want to send objects over networks.



- Ideally, remote method / function / procedure calls would be transparent to developers and appear as if things were happening locally.
- Objects created in one language need to be interpreted and usable by other languages. This is known as marshalling and unmarshalling.

Doing this with sockets and packets can get verbose and complicated.

Object Oriented Approaches: Java RMI

Designed to make communication between multiple Java programs, running in separate JVMs, as much like making a method call inside a single process as possible.

Uses a **client-side stub**:

- An object in the client's JVM to represent the server-side object.

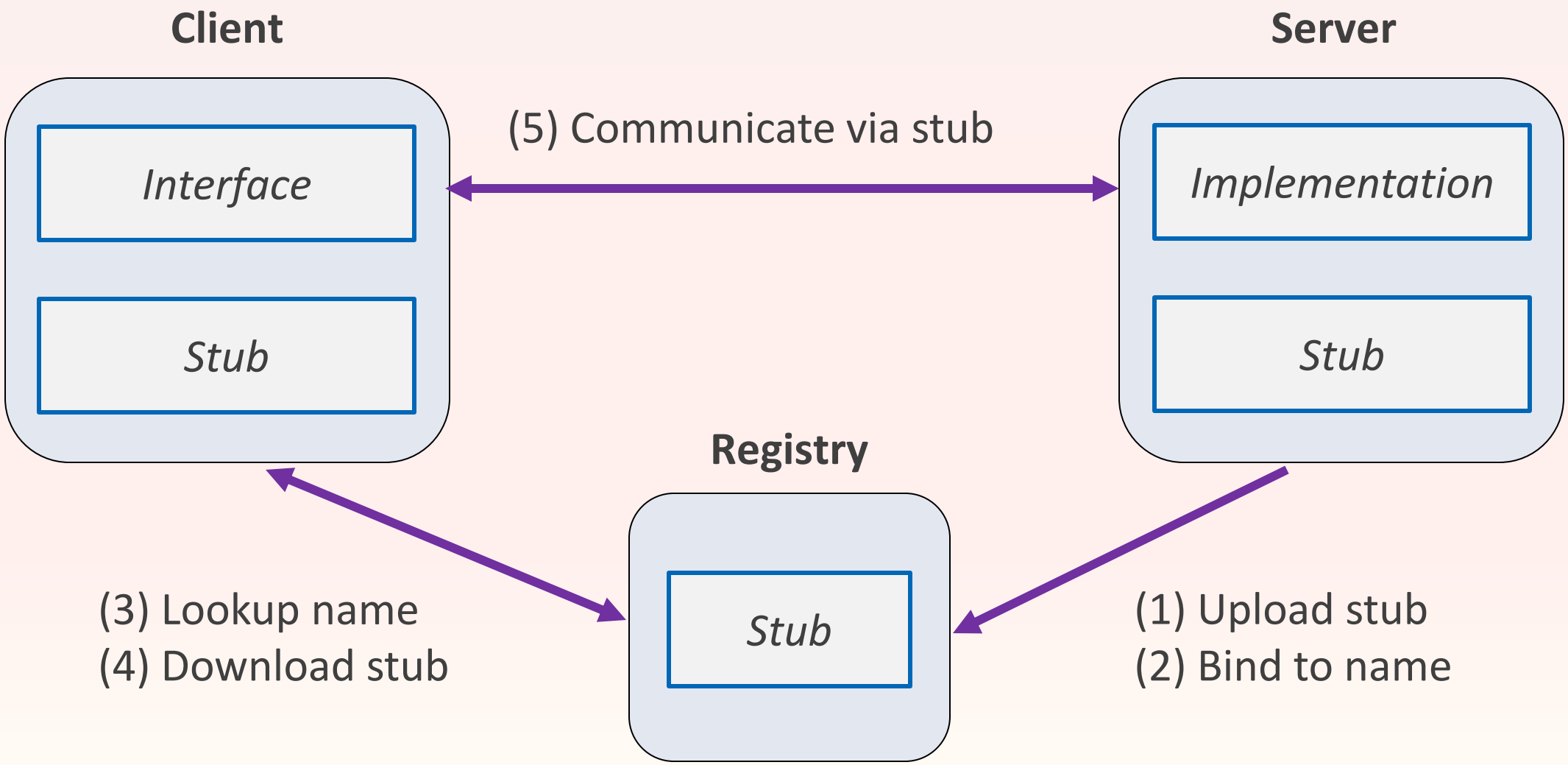
Uses a **server-side stub**:

- Object maintaining network connections, marshalling and unmarshalling data on server side.

Central naming service called rmiregistry:

- Functions like a phone book, providing the server associated with a naming service.

Object Oriented Approaches: Java RMI Stages



Object Oriented Approaches: RMI Concerns

- Java RMI was useful – it is truly Object Oriened and works as expected.
- Servers could run any code sent by the client, and vice versa. But...
- Registry isn't necessary for things to work, and it isn't managed for you.
- It was essentially Java-only. Not great for platform independence.
- Both client and server JVMs must be compatible.
- RMI allows you to send runnable code, not just messages, across machines.



Concept Check: Remote Code Execution

What problems might occur if we can send arbitrary runnable code over the internet?



Object Oriented Approaches: (In)secure RMI

- Arbitrary Remote Code Execution is quite literally a security nightmare.
- Sending runnable code over the internet is dangerous!
- There were security measures in place, but many people didn't use them.
- These things also tend to get blocked by firewalls.
- Which RMI was aware of... and would try to disguise itself as web traffic.

- Generally, a very bad idea outside of private, closed networks.



Object Oriented Approaches: What about us?

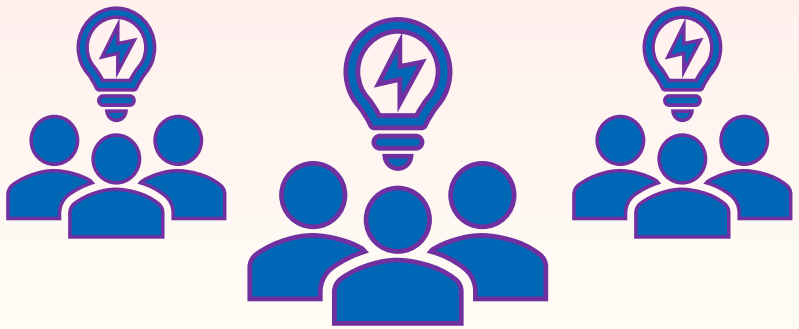
We no longer do Object Oriented distributed programming.

But approaches have been designed to look like we still do.

Programmers need to be aware:

- What you see is not always what you get
- Expectations may clash with reality

Over the coming weeks we see how we can send data, not code, over the internet in a way that mostly aligns with a familiar style of programming.



The Web: Origins of the World Wide Web

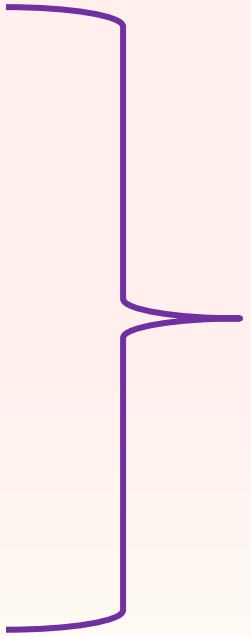
Invented by Tim Berners-Lee at CERN (1989) to accelerate structured information sharing.

Resulted in three core web technologies:

HTML (Hyper-Text Markup Language)
- How to correctly show data.

URL (Uniform Resource Locator)
- How to find that data.

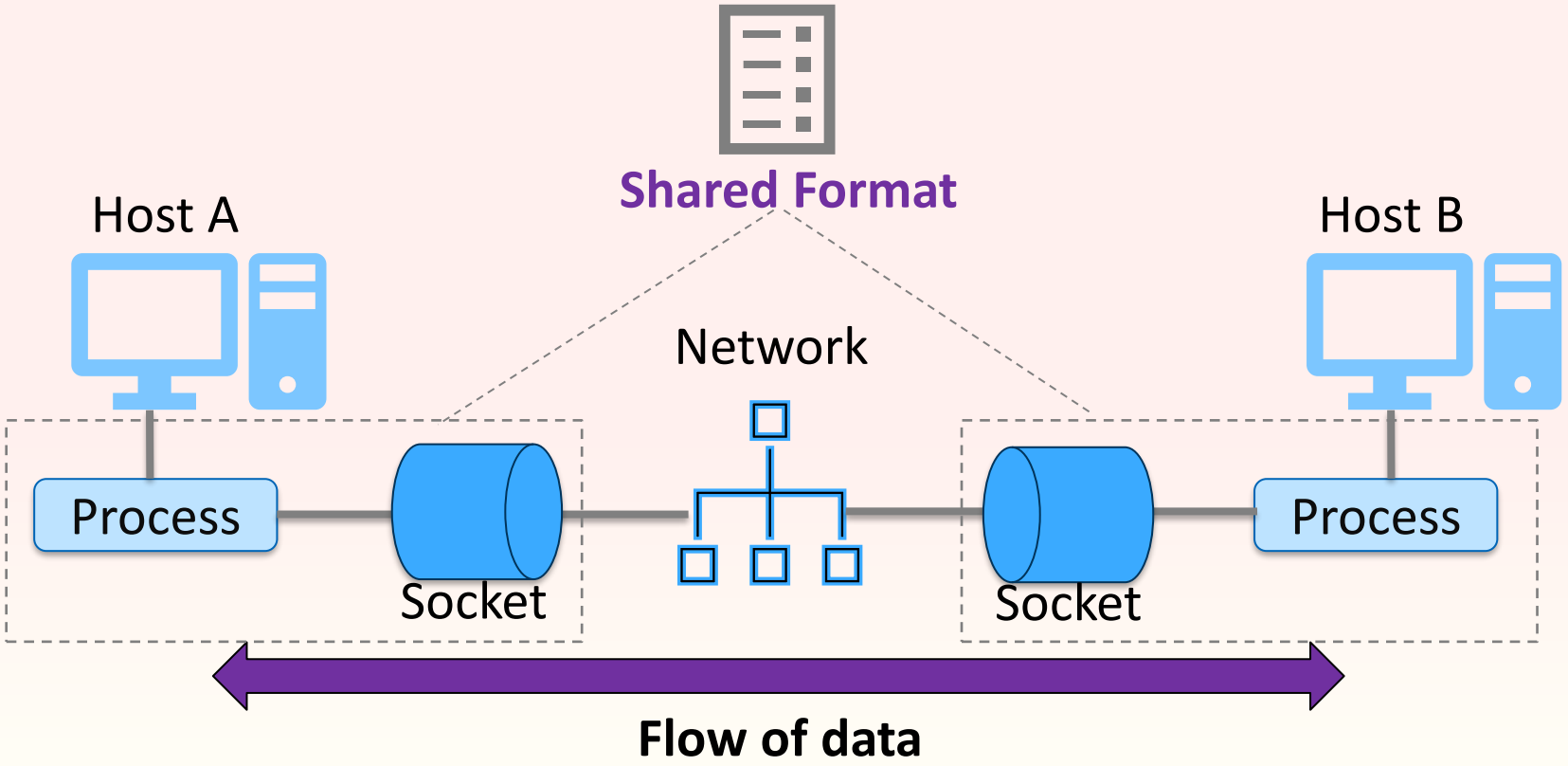
HTTP (Hyper-Text Transfer Protocol)
- How to move data from one machine to another.



We'll look at each of these in turn.

The Web: Motivating Markup Languages

- Standardised description of structure and meaning of arbitrary data.
- Described using (mostly) tags and elements.
- Interpretation becomes platform dependent.
- Separates content and presentation.

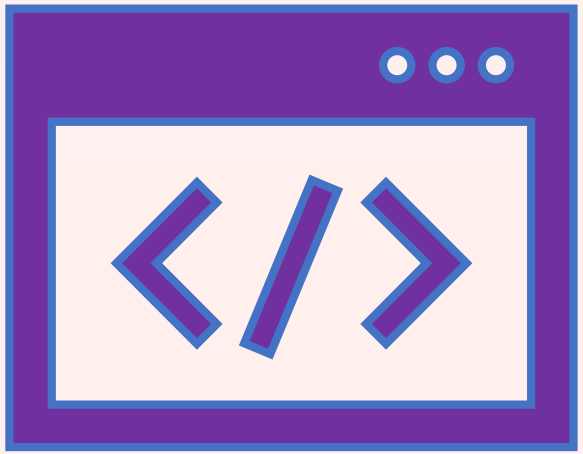


The Web: HTML

Markup system composed of:

- Elements
- Attributes
- Tags

which represent specific content.



- HTML tags provide semantic meaning
- Do not specify or impose logic or describe behaviour.
- Defines *what* is meant to be parsed by the browser.
- Does not concern *how* content is presented.

The Web: Uniform Resource Identifiers / Locators

Okay, so we can:

- *Present* data in a uniform way using markup languages.
- *Identify* machines and processes over networks using TCP/IP.

What about individual resources (documents, images, videos)?

- Berners-Lee proposes the Uniform Resource Identifier (URI).
- URIs are a superset of the Uniform Resource Locator (URL)
- URIs identify resources, URLs are how you locate them.
- All URLs are also URIs - they are also structurally identical...

The Web: URI / URL Format

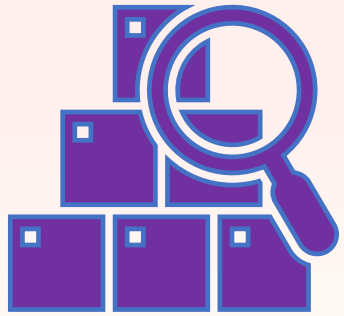
Scheme Authority Path

https://example.com:8080/store/login.html

What resource am I trying to access?

Which machine and process is hosting it?

Which **protocol** does it use to communicate?



The Web: HTTP Methods

They provide us the actions needed to interact with static resources.

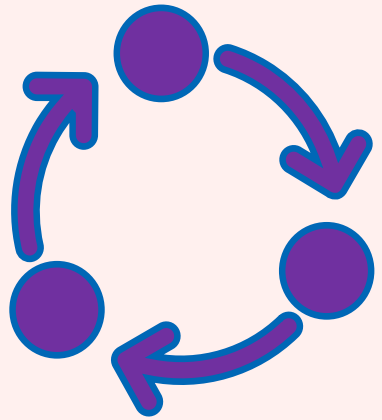
POST: Creates and pushes data to the recipient

GET: Reads data from the request recipient.

PUT: Updates some data held by the recipient.

DELETE: Deletes a resource held by the recipient.

HEAD: Requests checks for a resource without retrieving it.



There are others, but we won't be using them.

The Web: HTTP Responses

Each HTTP request is met with an HTTP response.

The nature of that response is "summarised" by a 3-digit response code.

There are many different codes for different purposes.

Codes fellow a pattern and are classified according to the first digit.

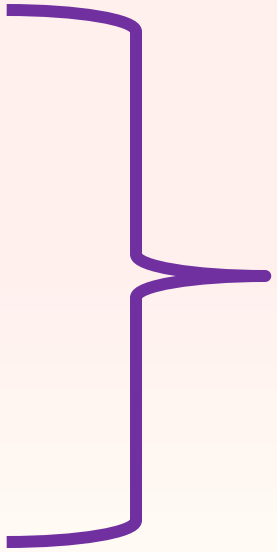
1 - (Information)

2 - (Success)

3 - (Redirection)

4 - (Client Fault)

5 - (Server Fault)



Very useful for conveying information in terms of client-server interaction.

The Web: HTTP Headers

Each HTTP requests/responses includes a section called the header.

This contains a lot of protocol and application specific data.

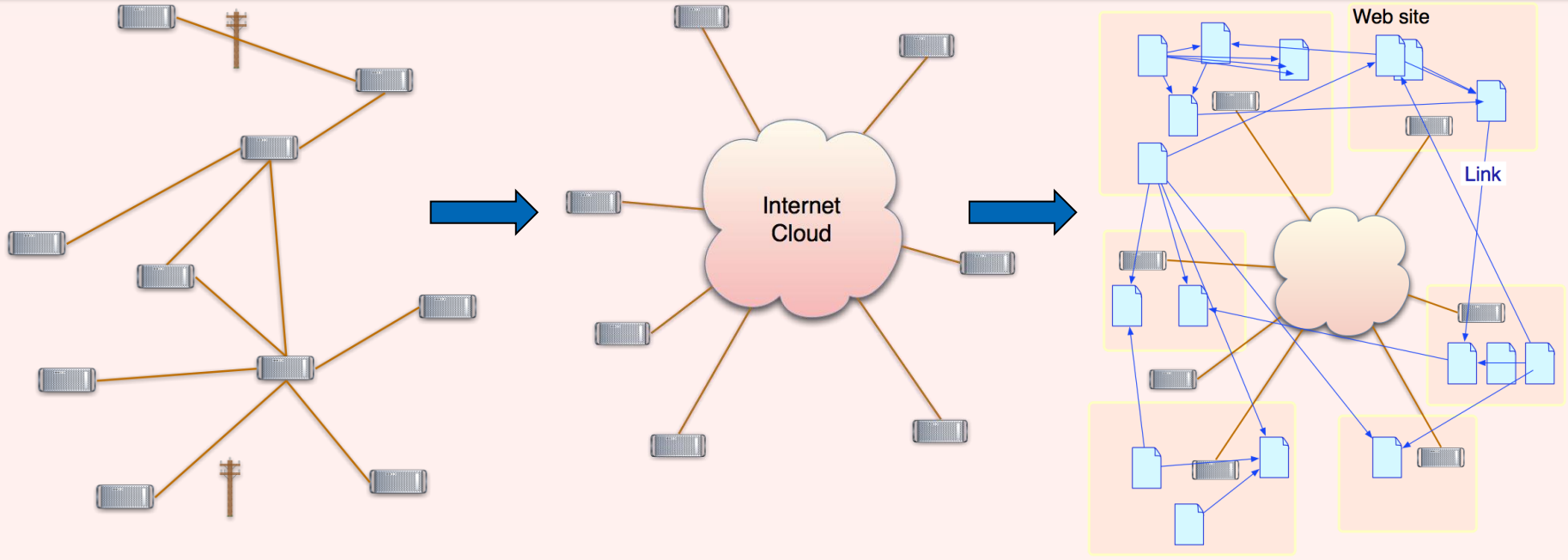
That data uses a "Name : Value" format.

```
Host: swansea.ac.uk
Accept: text/html,application/xml,application/json
Date: Wed, 9 Oct 2024 14:34:08 GMT
```

- Many of these are optional.
- Some describe the data the underlying HTTP body is carrying.
- Not the only way to pass parameters between client and server

The Web: WWW and Distributed Computing

Wired Machines → Files between Machines → Interactive documents

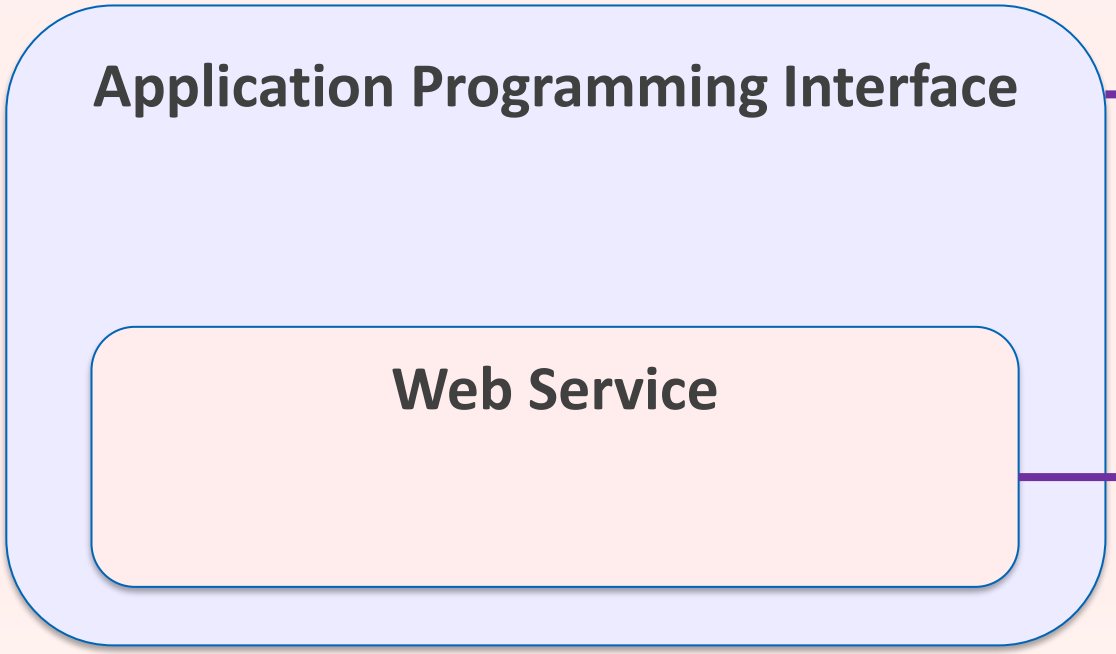


Courtesy: Tim Berners Lee

HTML empowered exchange of information.

But what about exchange of services?

Web Services: Web Service or API?



Functions and procedures between any two software applications.

Type of API that operates via web communication.

Web Services: Desirable Properties

Open Infrastructure:

- Promote interoperability among services.
- Use industry-standard, vendor-independent protocols (HTTP, XML)
- Piggyback on existing networking, data formatting, security etc.

Language Transparency:

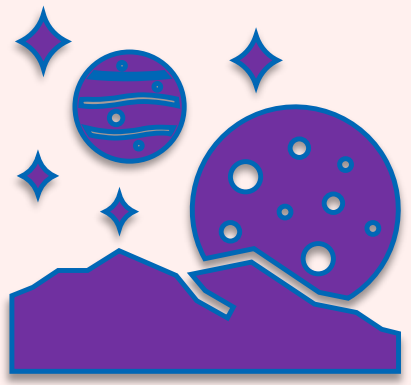
- Clients and services can interoperate irrespective of their programming language or underlying OS.

Modular Design:

- Meant to be modular in design so that new services can be generated through the integration and layering of existing services.

Web Services: Landscape

- HTTP and URLs allow clients to retrieve HTML pages from servers.
- Unlike the RMI/CORBA/DCOM model, calling specific functions was not not possible.
- Exchange of data was possible to an extent with XML (eXtensible Markup Language).
- Running specialised services was still limited.



In come web services:

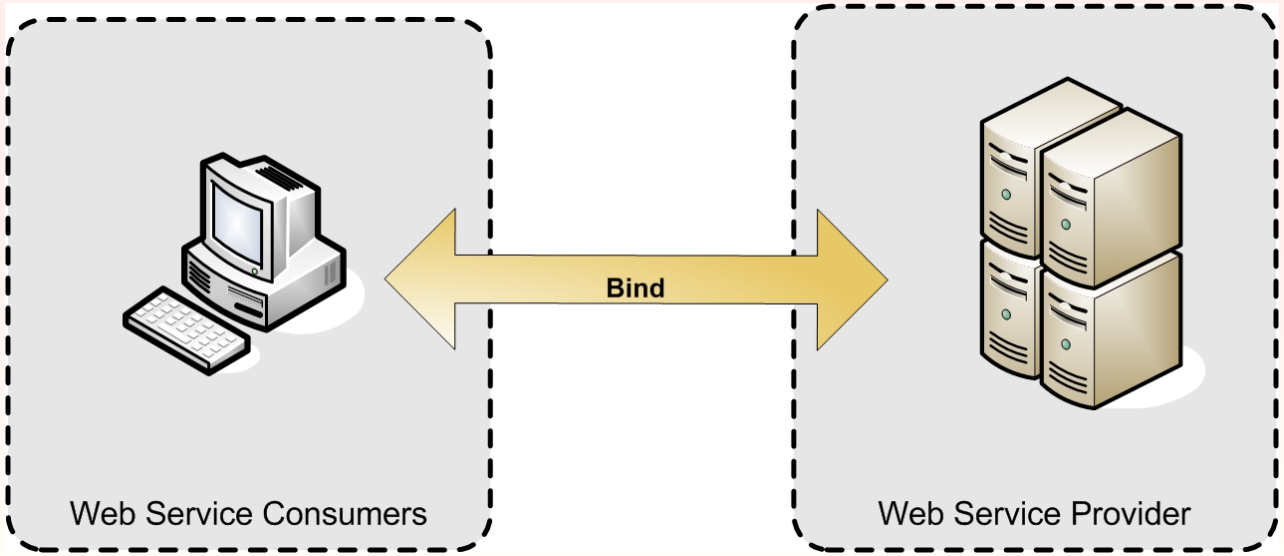
- Application-specific clients can interact with a service via a specialised interface using existing network architecture and web technologies.

Web Services: The (General) Architecture

The simplest Web service system has two participants:

- A service producer (provider)
- A service consumer (requester)

The provider presents the interface and implementation of the service, and the requester uses the Web service.



Web Services: The Basic Requirements

Message passing framework:

- Handling errors and faults in application level to ensure fault tolerance.

Intermediaries:

- Augmenting message with metadata and ways to monitor, route, or transform messages.

Common interface definition:

- Common way to expose a set of operations/services.

Easy location of services:

- Organising the web services via registries to ensure that they are found easily by users.

Web Services: The Service Orientated Architecture (SOA)

Designing a software system that

- Is composed of largely independent services.
- Uses standard protocols to communicate data.
- Publishes interfaces to support interoperability
- Can be distributed over networks.
- Aims to make its services extensible.

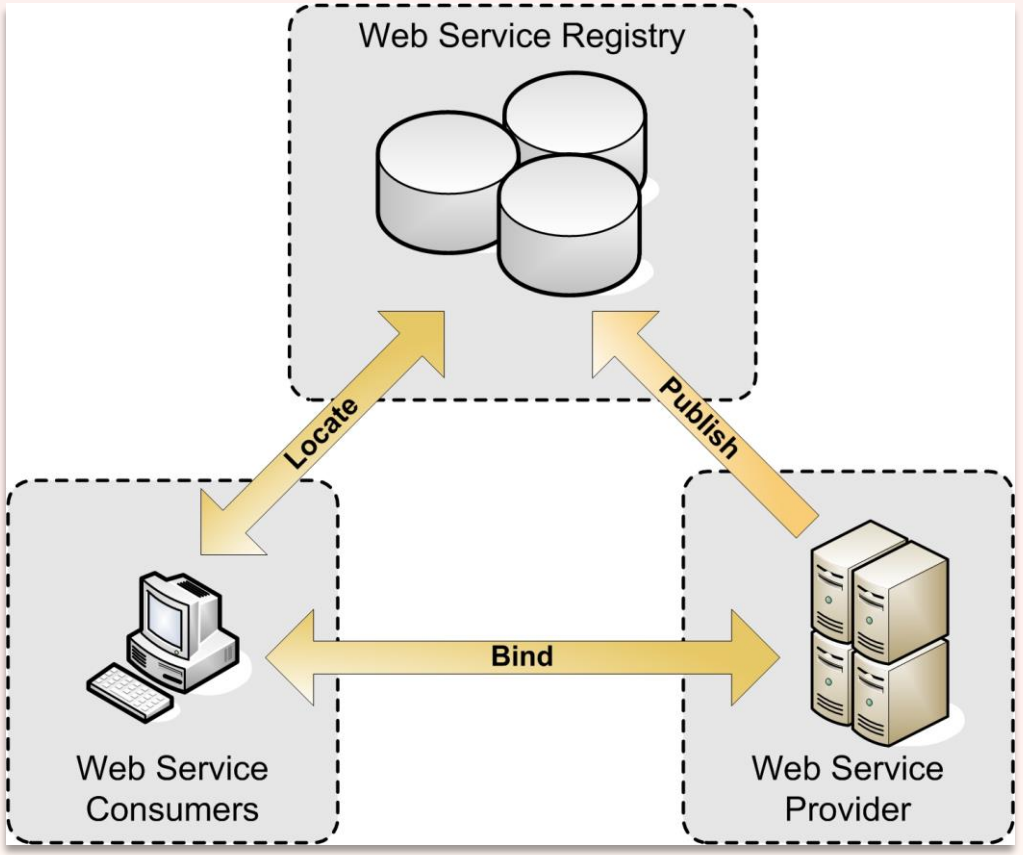
A web service is one of the most common instances of a SOA implementation.

Web Services: The Service Orientated Architecture

A more sophisticated system:

- A registry, acts as a broker for Web services.
- A provider, can publish services to the registry
- A consumer, can then discover services in

So how did developers agree to design this?

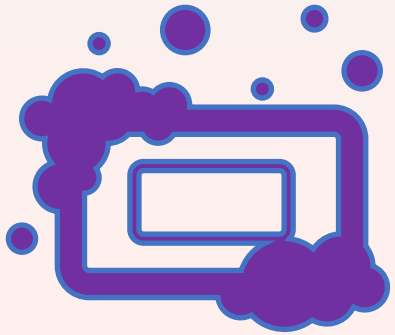


Web Services: SOAP

Originally stood for Simple Object Access Protocol.

Later, Service Orientated Architecture Protocol.

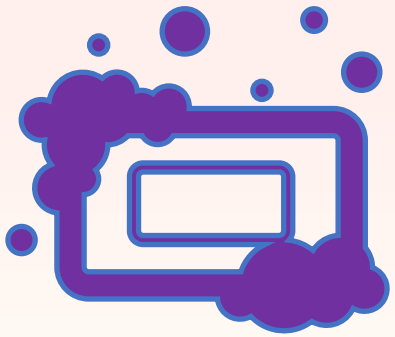
Now it just means SOAP...



- An early attempt at meeting this specification, starting at Microsoft, in the mid-late 1990s.
- Uses XML messages, or envelopes, to encode method calls and pass parameters.
- Uses HTTP (and other transport protocols) to exchange messages.
- Can pass information describing how data should be interpreted

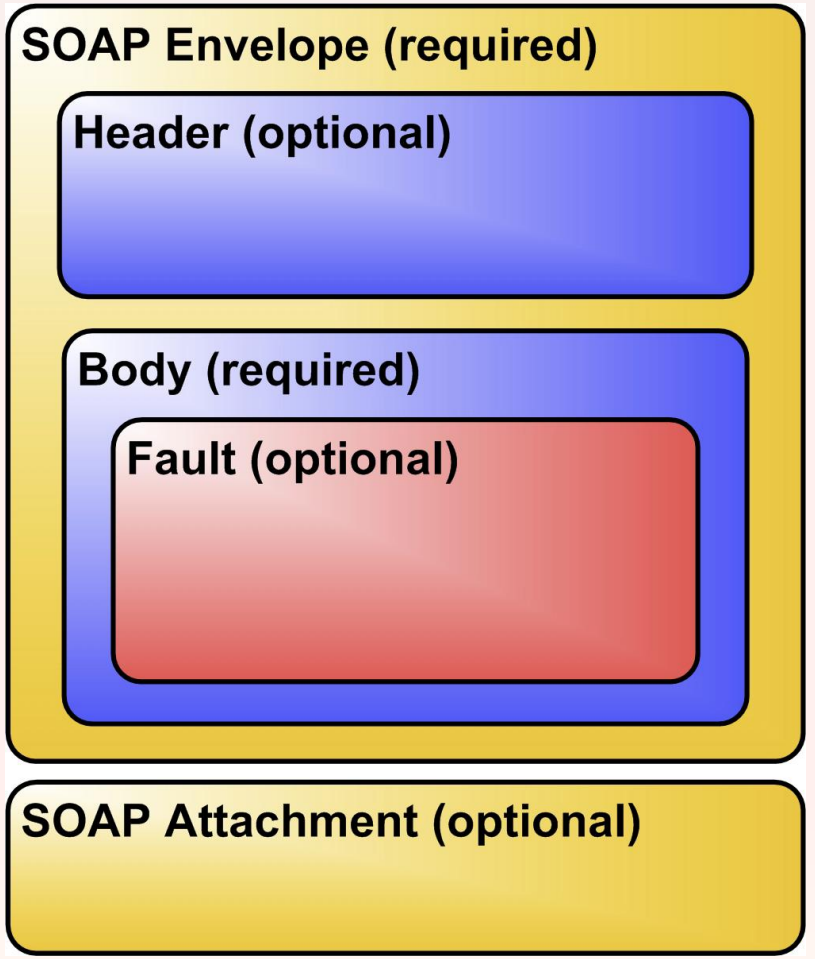
Web Services: The Good SOAP

- XML became well-established, meaning tools for parsing and generating.
- XML messages also appeared to be 'legitimate' traffic, reducing the security concerns (but not completely preventing them)
- No executable code is being passed, just information for RPCs to work.
- Is truly platform independent and means client-server applications could be loosely coupled.



Web Services: The SOAP Message

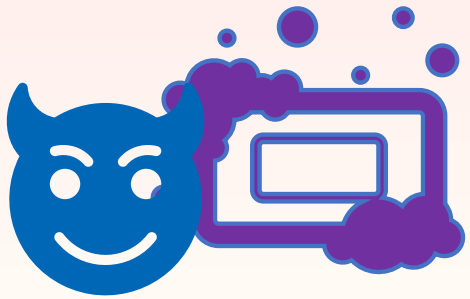
Element	Description	Required?
Envelope	Top-level XML element	Yes
Header	Metadata about SOAP message	No
Body	Data sent for receiver to process	Yes
Fault	Any Exceptions and errors	No



Web Services: The Not So Good SOAP

SOAP works fine but:

- XML messages are verbose.
- Not the most pleasant things to look at.
- Message sizes mean they're not computationally "cheap" to process.
- The way data is passed and the way methods are called are separate.
- Operations, data types, parameters need a client-server contract



Web Services: The REST Alternative

Representational State Transfer (REST)

Focuses on using HTTP, URIs and a simpler markup language (JSON):

- No longer interact with services, but with resources.
- URIs identify the endpoints for those resources (and parameter passing)
- HTTP methods describe the operations you can perform
- HTTP codes and headers can carry additional information
- JSON much simpler to write, read and less verbose.
- Also supports XML



Web Services: Resources

Consider the URI:

`https://www.mywebsite.co.uk/stuff/morestuff/thedeepeststuff`

Each aspect of the URI indicates a resource held by the server.

REST allows URIs to represent a resource and associated parameters.

`https://www.mywebsite.co.uk/resource/parameter1/parameter2`

The HTTP method used with that URI then defines what operation will be performed on the server, using those parameters, for that specific resource.

Web Services: REST Request

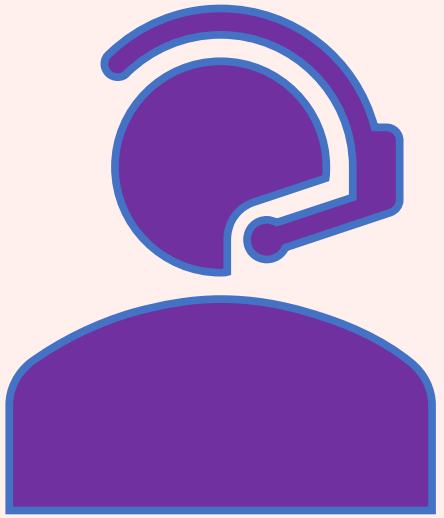
```
GET /users/12345 HTTP/1.1
Host: www.example.com
Accept: application/json
```



Yes, that's quite literally it.

Web Services: REST Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: length
{
  "userId": 12345,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com"
}
```



DEMO

Postman and REST

Web Services: SOAP and REST

SOAP



A **protocol** following RPC architectural style.

Must represent messages in XML.

Relies heavily on HTTP for transferring the XML payload

REST



An **architectural style**

Supports multiple data representation languages.

Uses HTTP protocol and URI for methods and scoping information.

WEEK 4

RESTful Principles and Spring Boot

RESTful Architectural Style

Roy Fielding's Dissertation on RESTful Architectural Style [1].

Basis for REST APIs.

6 Constraints:

a) Uniform interface

b) Stateless

c) Cacheable

d) Client-Server

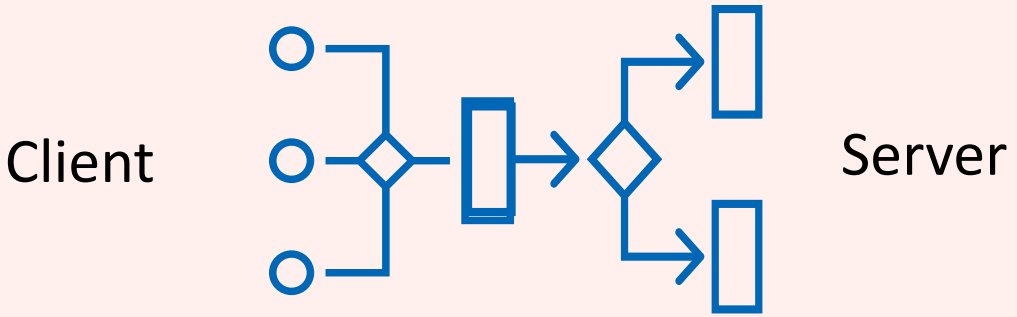
e) Layered System

f) Code on Demand

[1] http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

REST: Uniform Interface

Defines the interface between clients and servers.



Simplifies and decouples the architecture.

Dictated by four guiding principles of the uniform interface.

REST: Guidelines for Uniform Interfaces

Resource-based:

URIs identify individual resources.

Manipulation of Resources through Representations:

Methods and Representations can change resources.

Self-descriptive Messages:

Each message should describe how it should be processed.

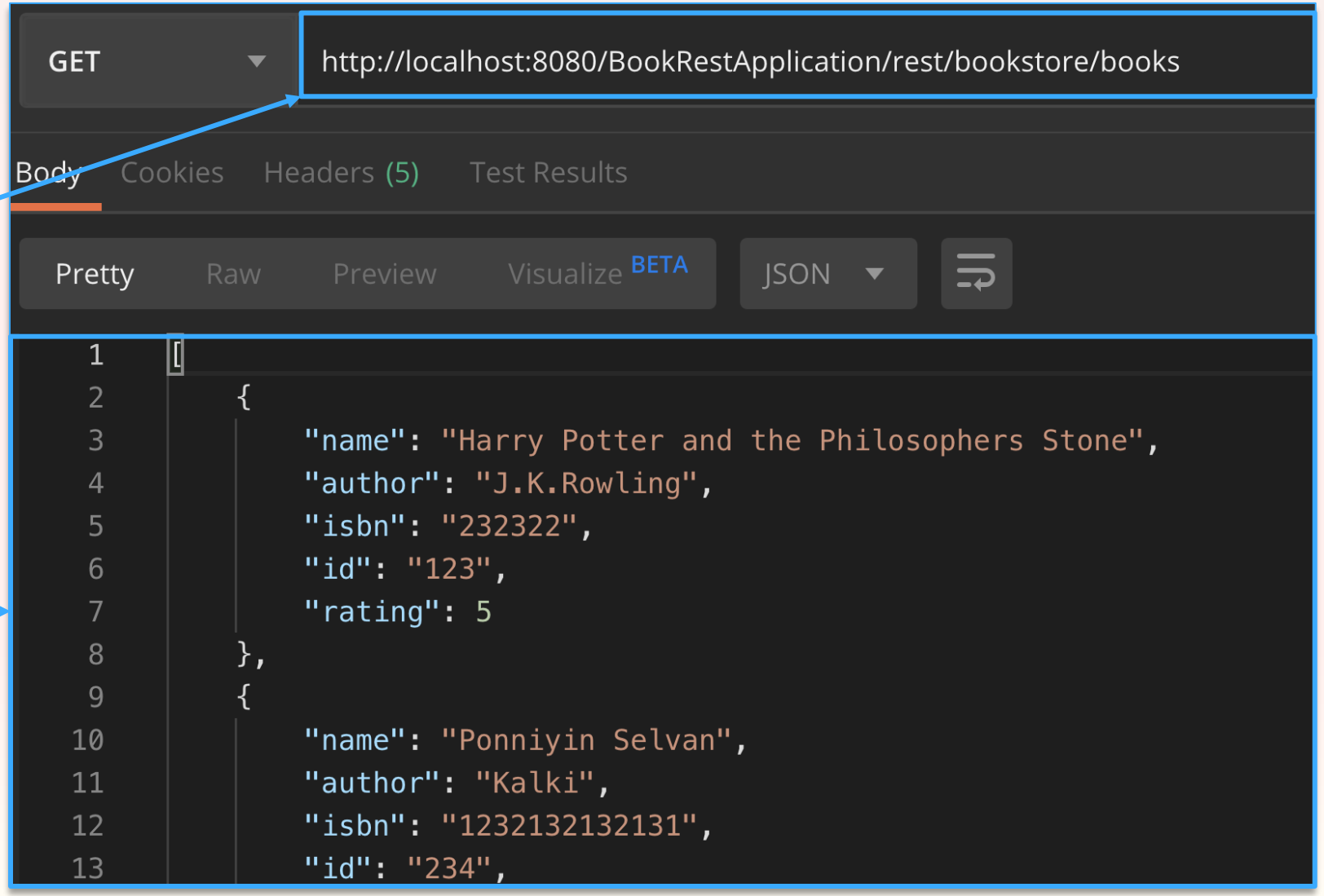
(HATEOS):

Hypermedia **a**s **t**he **E**ngine **o**f Application **S**tate.
Links to related resources are included in messages.

REST: Resource vs. Representation

Resource
URI.

Representation:
Plaintext
JSON list



JSON: Specifications

Introduction to components of the language:

- https://www.json.org/json-en.html



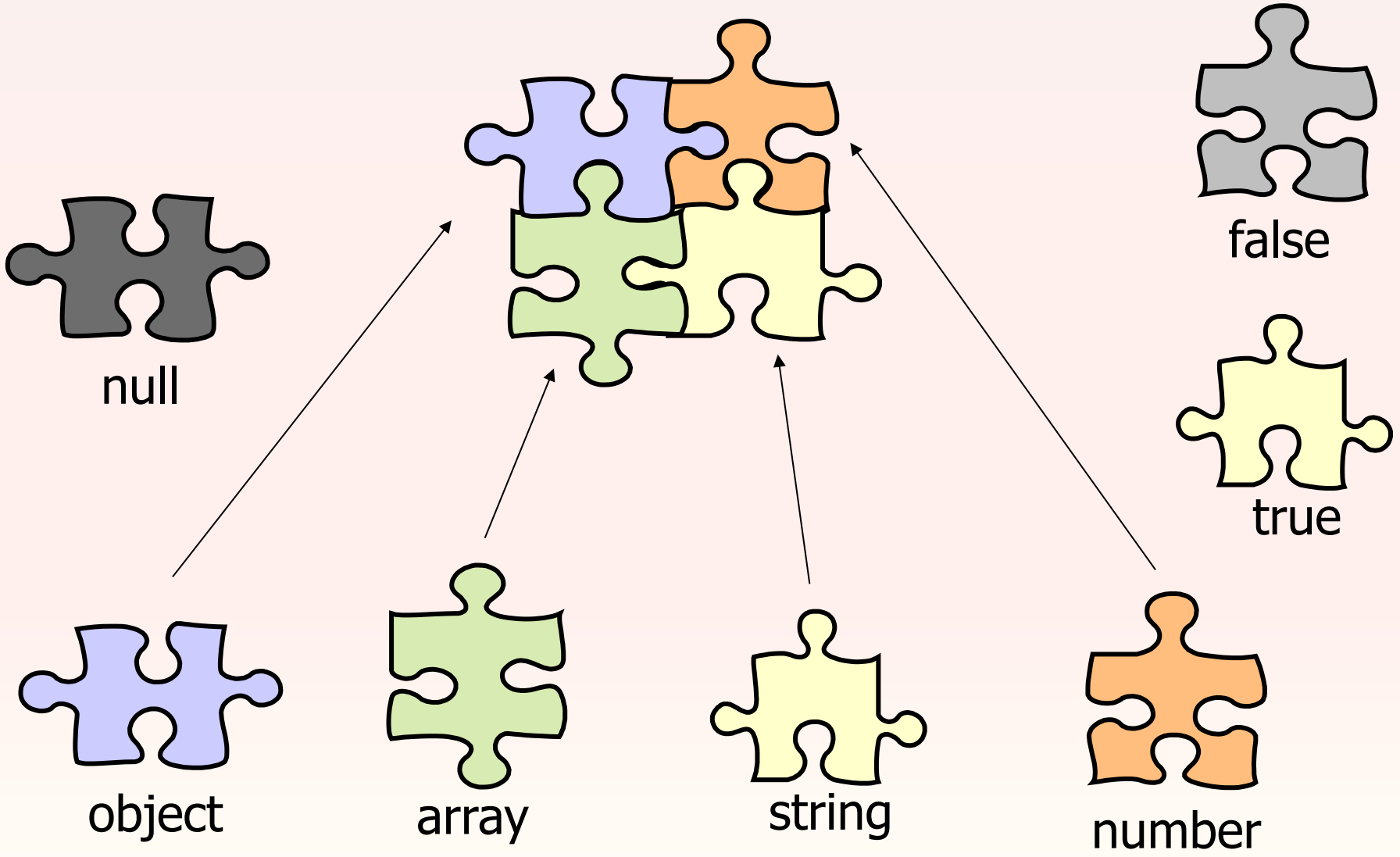
Formal specification of the language:

- https://www.rfc-editor.org/info/rfc8259

Two specifications for JSON-Schema

- https://json-schema.org/draft/2020-12/json-schema-core
- https://json-schema.org/draft/2020-12/json-schema-validation

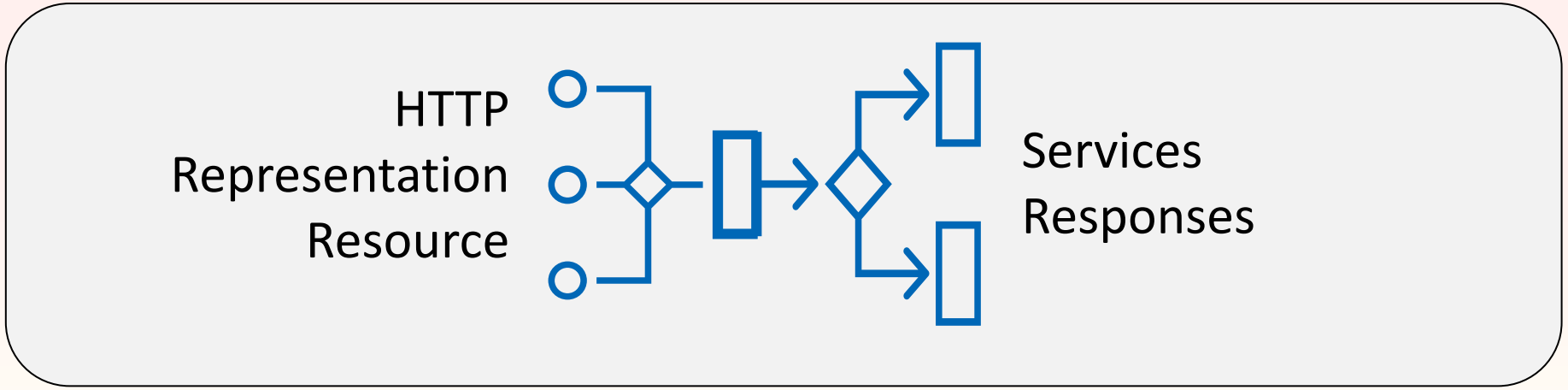
JSON: Seven Simple Components



REST: Stateless

Stateless means information necessary to handle the request is contained within the request itself.

Whether as part of the URI, query-string parameters, body, or headers.



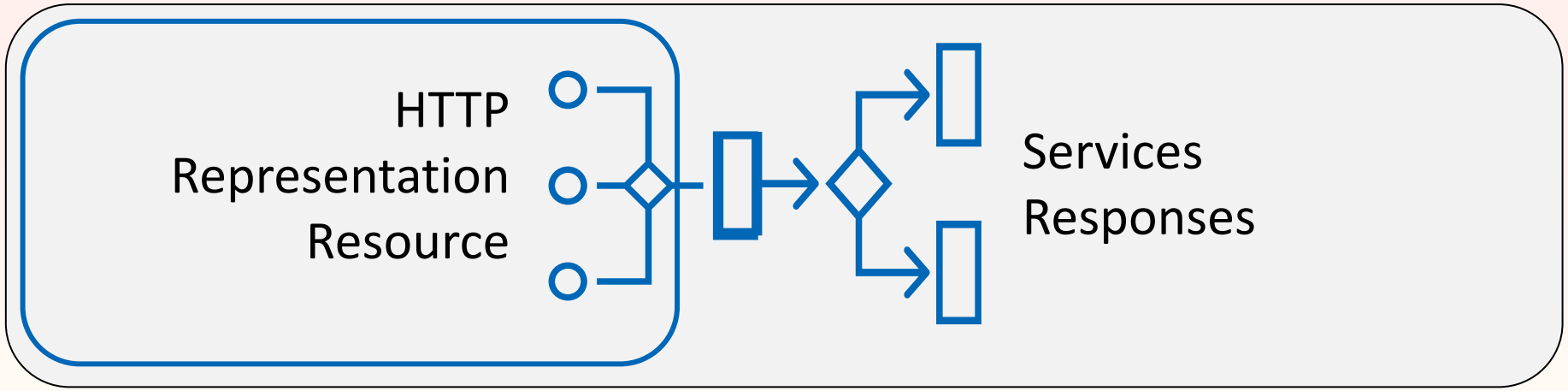
REST: Stateless (Application)

Application State:

The information needed by the server to **fulfil** a request.

- i.e necessary for the **current session** or **request**.

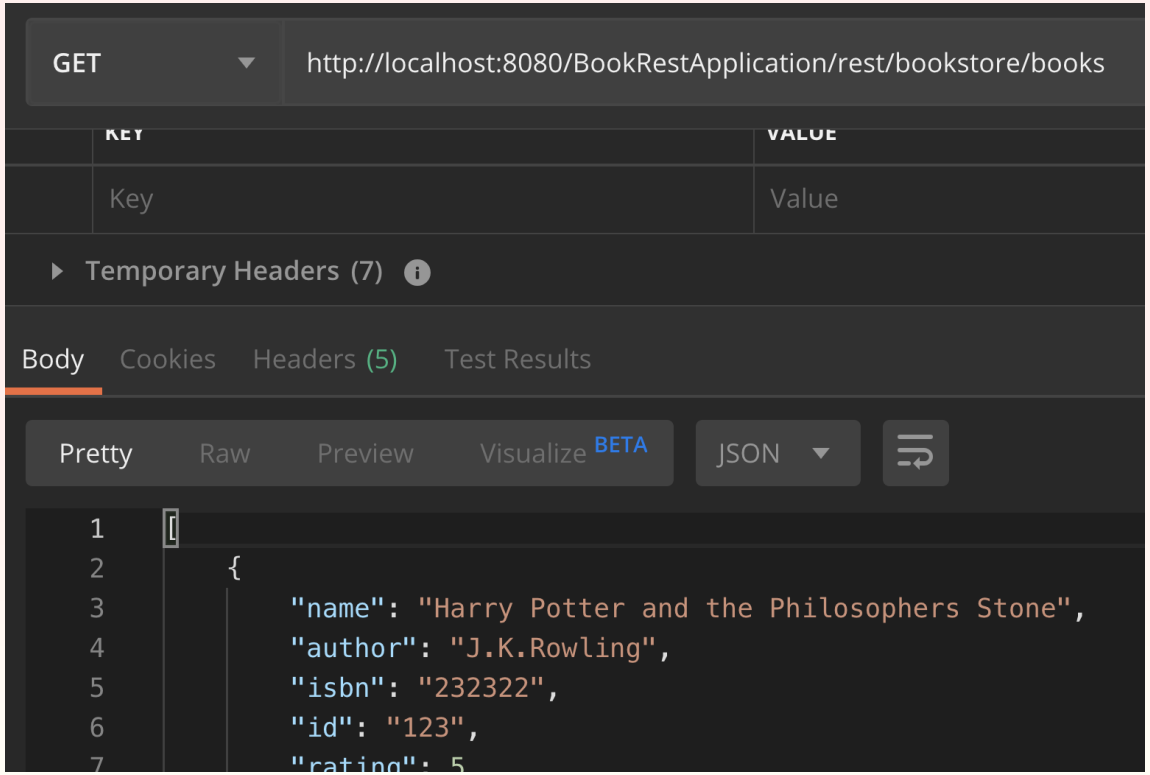
Consider application state as the data that could **vary by client**, and **per request**.



REST: Stateless Example - GET

GET on a Resource:

- Client retrieves all books from server
  (no state change happens on the application)
- JSON representation of the resource sent as response.



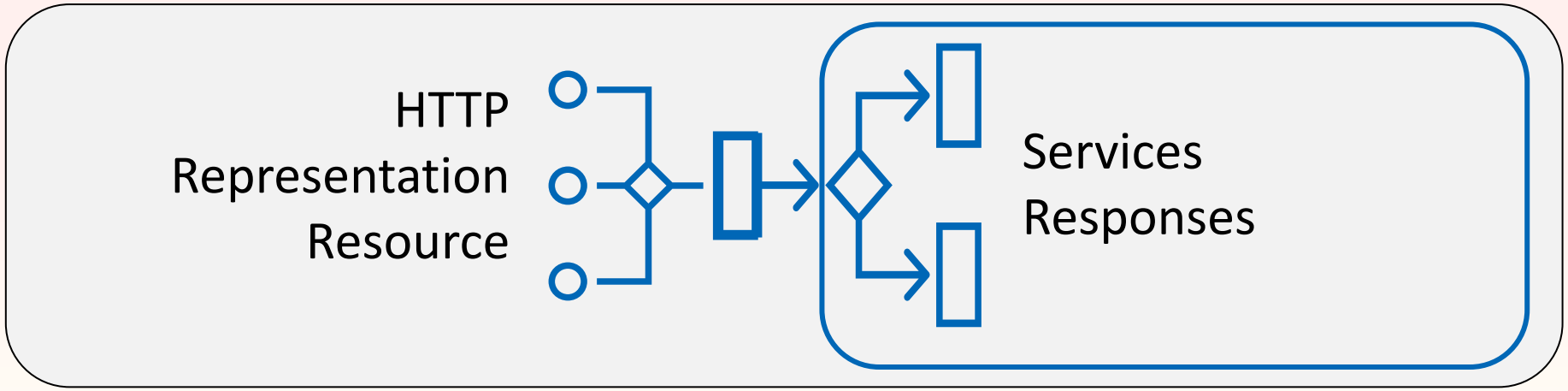
REST: Stateless (Resources)

Resource State:

The data that defines the resource representation.

- i.e the consistent responses sent across different clients.

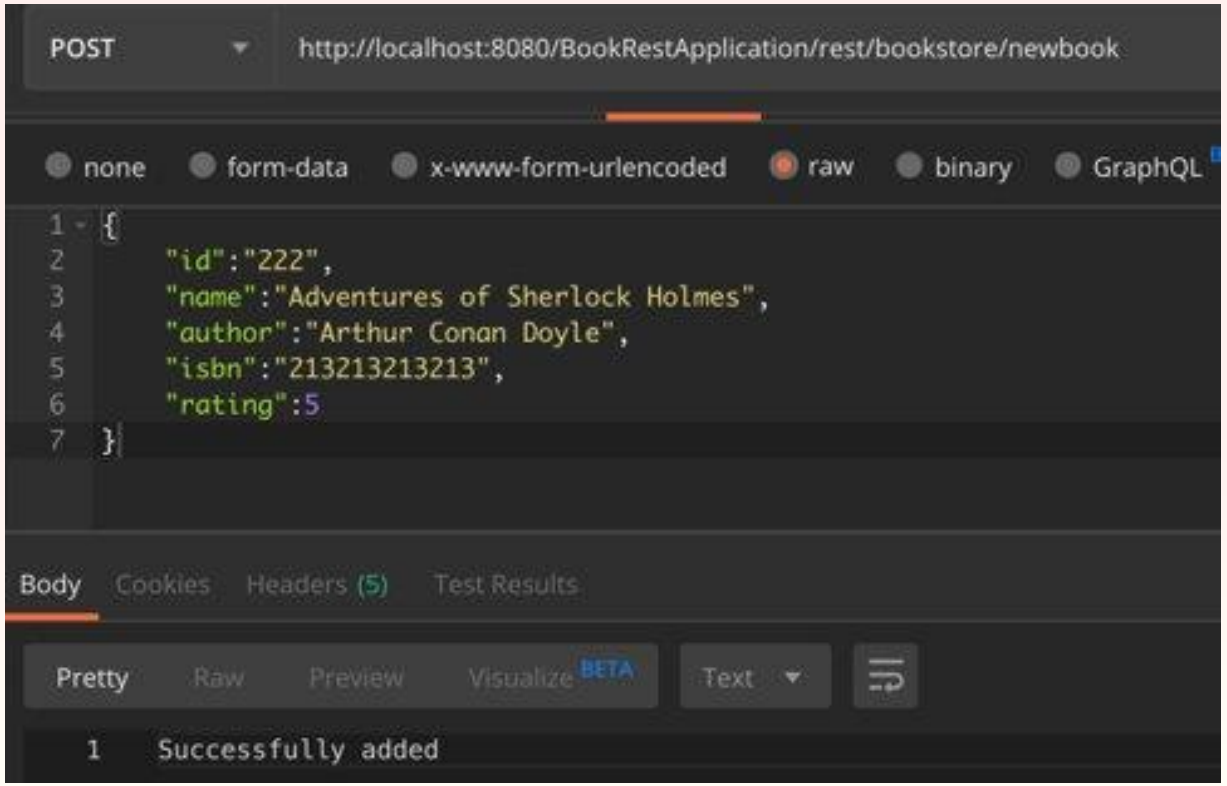
For instance, the static data stored in service database.



REST: Stateless Example - POST

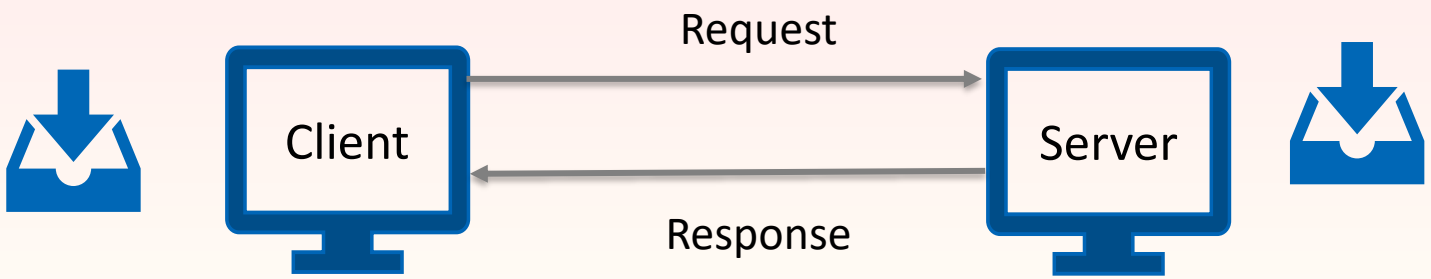
POST on a Resource:

- Client sends new book details in JSON representation.
- Tells the server to add new book to the resource thereby **changing** the **resource state**.



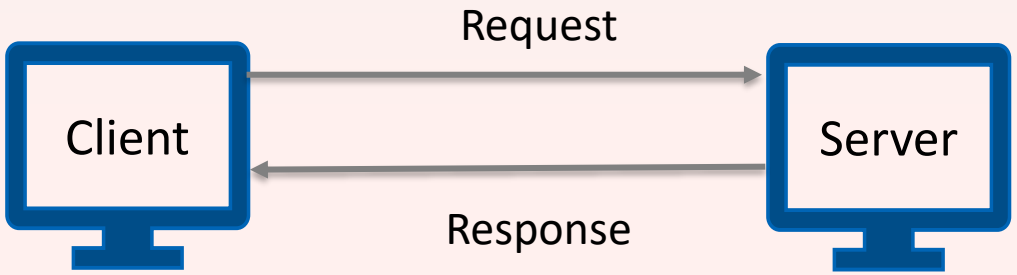
REST: Cacheable

- Clients can cache (remember) responses.
- Responses define themselves as cacheable, or not.
- Prevents clients reusing stale or inappropriate data.
- Well-managed caching (partially / completely) eliminates client–server interactions.
- Improving scalability and performance.



REST: Client-Server

Uniform interface separates clients from servers.



Not concerned with data storage.

Manages user experience

Client code more portable.

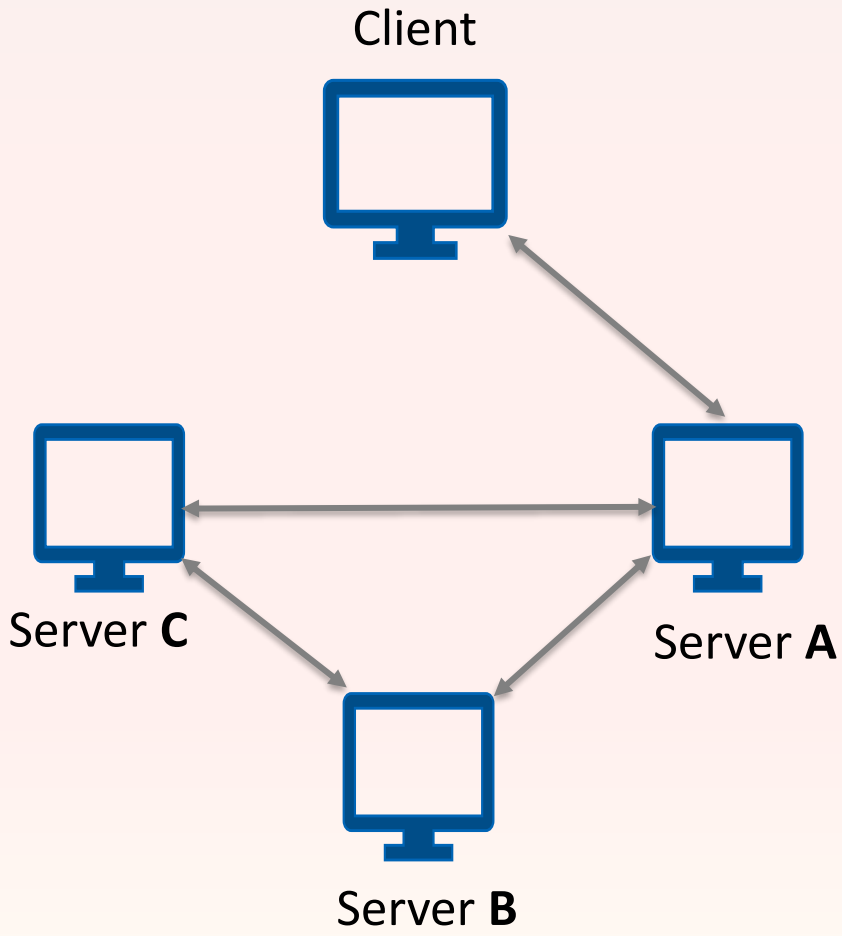
Not concerned with user interfaces or user states.

Manages resources and responding.

Server implementation simpler and more scalable.

REST: Layered System

- Client sends request to endpoint.
- Multiple servers can process a request.
- Layered system can help delegate work, contributing to servicing clients.
- Layers may also enforce security policies.



REST: Code on Demand (optional)

- Servers can offload computation to clients.
- Can extend functionality.
- Not widely practiced, can be useful.

Most commonly:

- Client-side scripts (JavaScript).



REST: Constraints

Complying Constraints → REST

Enables distributed hypermedia systems:

- Performant
- Scalable
- Simple
- Modifiable
- Discoverable



NOTE:
Services violating non-optional constraints are **not** strictly RESTful.

HTTP Methods: Idempotency

Idempotent Method:

Can be called several times and outcome will not change.

- GET
- HEAD
- PUT
- DELETE

are idempotent for different reasons.

In contrast, POST is not idempotent.

Calling POST multiple times can have different outcomes.

HTTP Methods: Safety

Some methods have side-effects.

Others do not – we call these Safe methods.

Safe HTTP Methods:

- GET and HEAD (read only)

Not so safe HTTP:

- PUT usually *creates* a resource on the server.
- DELETE will *delete* the resource on the server.
- POST will *change* the resource on the server.

Representations: XML or JSON?

We can offer both:

- JSON is highly favoured and an informal convention (guess why).
- XML standards are mostly ignored (beyond syntax).
- Namespaces are not, nor should they be used in a RESTful service context.
- XML returned is more JSON like, simple and easy to read

Representations: Requests

Clients form a representation when sending requests.

Services must consider what data to support in these requests.

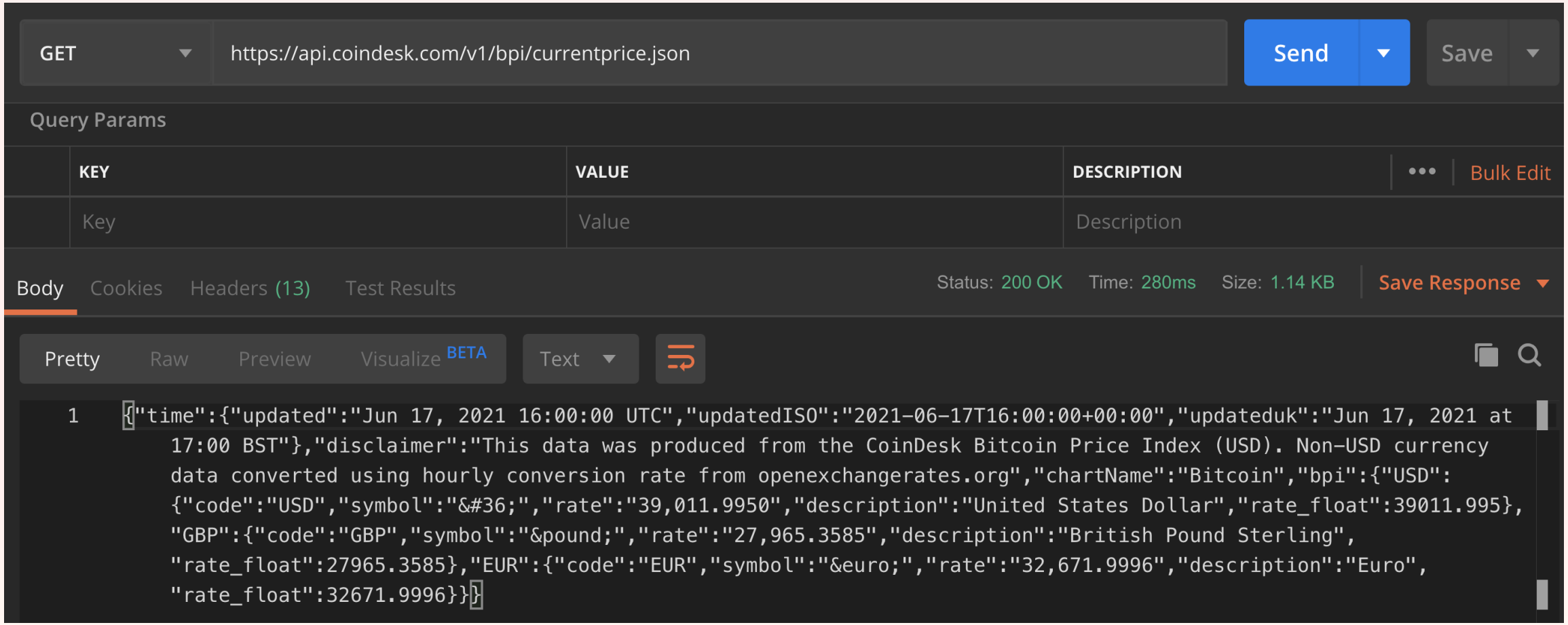
- Accept header
- File-extension-style format specifier
- Query-string parameter

etc.

```
GET /api/users/123 HTTP/1.1
Host: example.com
Accept: application/json
```

And then there's the question of naming the resources...

Representations: Example URI



Resources: Naming

A RESTful URI should refer to a resource – a description of data.

What does this mean?

- Use nouns to construct the URI. Avoid verbs.
- HTTP methods describe the action. The URI does not.

OK

`https://api.video.com:8181/films/genres/horror/all`

Bad

`https://api.video.com:8181/readFilms/getGenres/horror/listAll`

Nice article:
https://medium.com/@nadinCodeHat/rest-api-naming-conventions-and-best-practices-1c4e781eb6a5

Resources: URIs and HTTP Methods

Consider an e-commerce service:

- order system with customers, orders, line items, products…

Create a new customer in the system:

```
> POST http://www.example.com/customers
```

Read a customer with Customer ID# 33245:

```
> GET http://www.example.com/customers/33245
```

Resources: URIs and HTTP Methods

Same URI applies for other methods…

Proposed URIs for products:

http://www.example.com/products

Creating a new product.

> **POST** http://www.example.com/products

Interacting with that product:

> **GET | PUT | DELETE** http://www.example.com/products/66432

Question: What about creating a new order for a customer?

Resources: Choosing URIs

One option might be:

> **POST** http://www.example.com/orders



And that could work to create an order, but it's arguably outside the context of a customer.

URI that would offer better clarity:

> **POST** http://www.example.com/customers/33245/orders



Now we know we're creating an order for customer ID# 33245.

Implementing Web Services: Two approaches

There are two approaches to defining and implementing web services

Contract-first:

Create a contract file, then create artifacts based on the contract agreed.

Contract-last/Code-first:

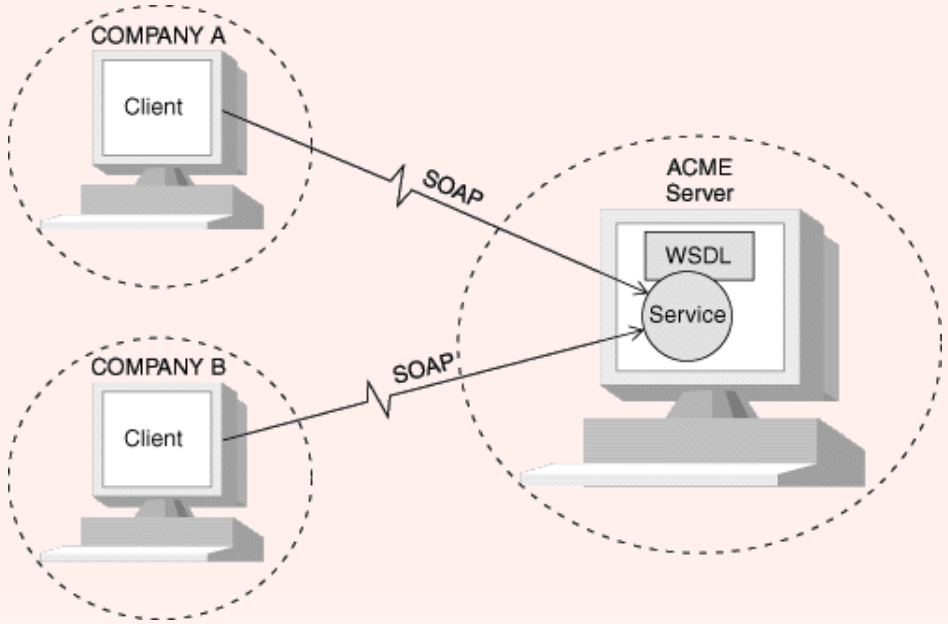
Create the Java interface first, then generate the contract from it.

Web Service Description Language: WSDL

Generally, web service contracts describe:

- The interface
- A set of supported operations
- Parameters for those operations
- Transmission protocol

in a standard format.



SOAP used something called Web Service Description Language (WSDL)

REST has multiple, but only two are really used – OpenAPI and RAML

Code First Approach: a.k.a Contract Last

Developers (typically) prefer to write code, then publish the documentation.

Most API frameworks support code annotations to do this automatically.

Ultimately, this can lead to a simpler pipeline:

- Server-side code is implemented and annotated
- Utilities read the annotations and generate the contract
- Utilities read the contract to generate client-side stubs

REST: Implementation in Java

JAX-RS

- Specification for REST implementation in Java
- Reference implementation in Jersey
- Requires external container for deployment.
- A lot of manual configuration

Spring Boot

- Alternative to the (complex) Java Enterprise specification
- Uses embedded container for deployment
- Adheres to a “Convention over configuration” principle
- Larger ecosystem of modules to build enterprise systems

Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Main entry point for the service.
Marked by @SpringBootApplication

Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Definition of service endpoints and "business logic".
Marked by @RestController, @RequestMapping, and many others

Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Configuration options for the service. Custom format to spring, very simple to understand.

Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Principle Maven file. Written in XML. Defines dependencies and project information.

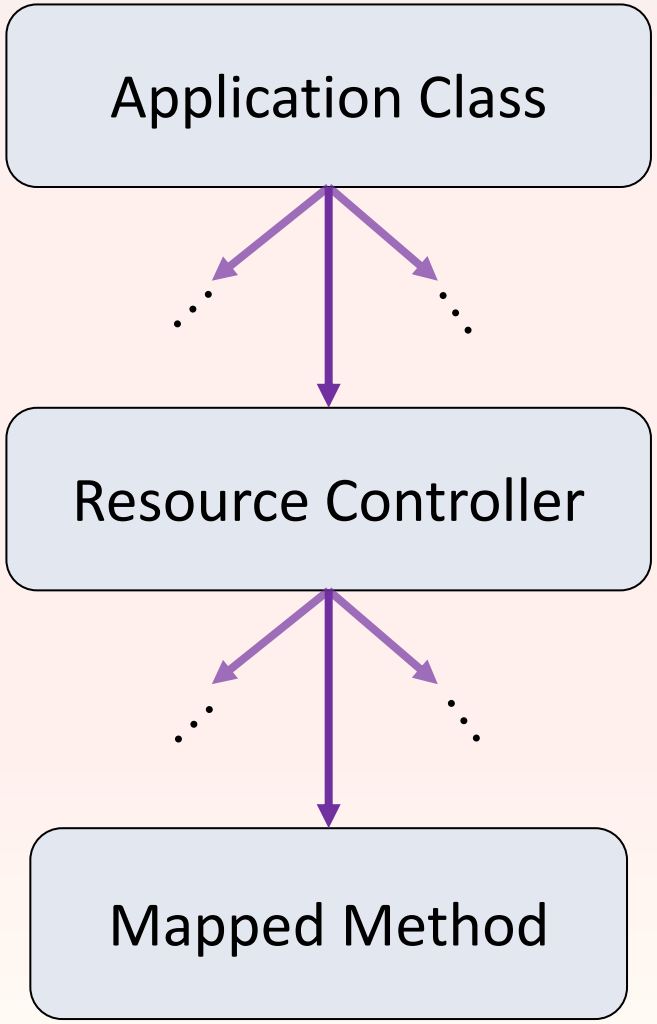
DEMO

Basic RESTful Web Service

WEEK 5

Parameter Passing and Response Building

Introduction: Resource Method I/O



- Defines the web service endpoint root
- Can manage application configuration

- Groups services under related endpoints
- Contains HTTP-mapped Java methods
- Provides default configs for incoming requests

- Define mapping of HTTP method to Java Method
- Define sub-resource endpoints
- Implements service “business logic”
- Provides control over responses

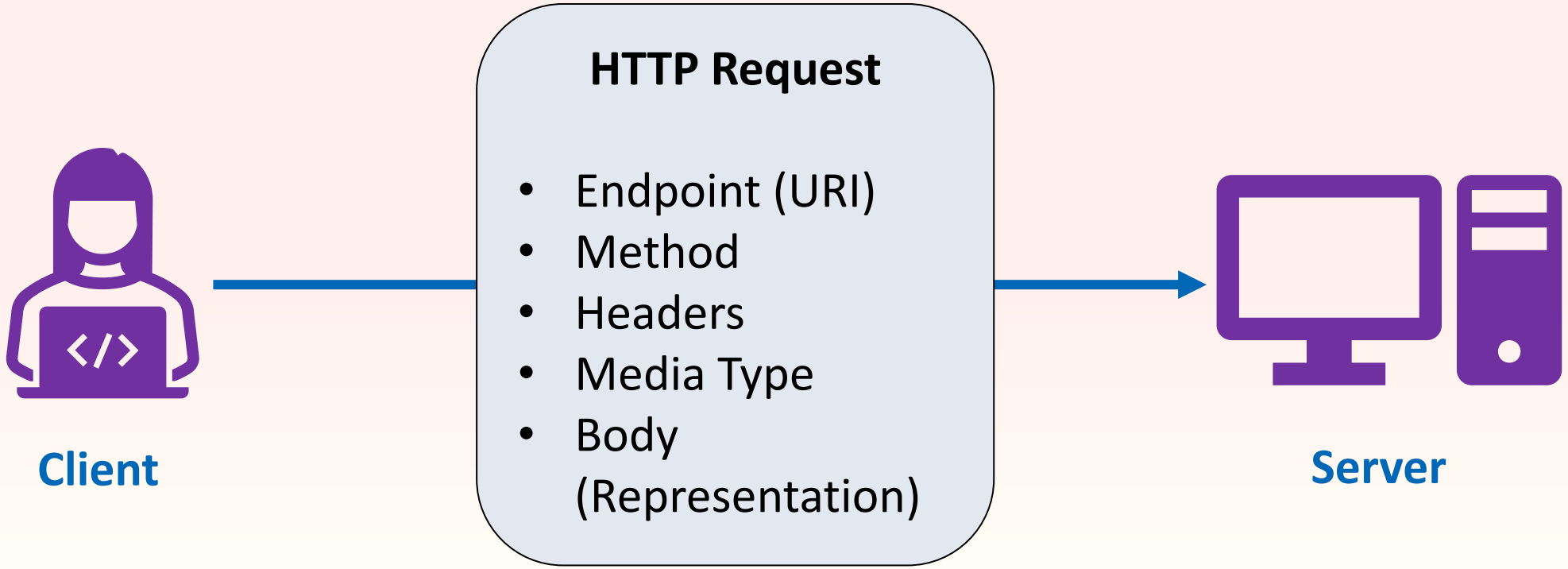
Parameter Passing: Customising Service Operations

A client sends a request to a web service. What does the request contain?

This data is used to fulfil the request. Is this information static?

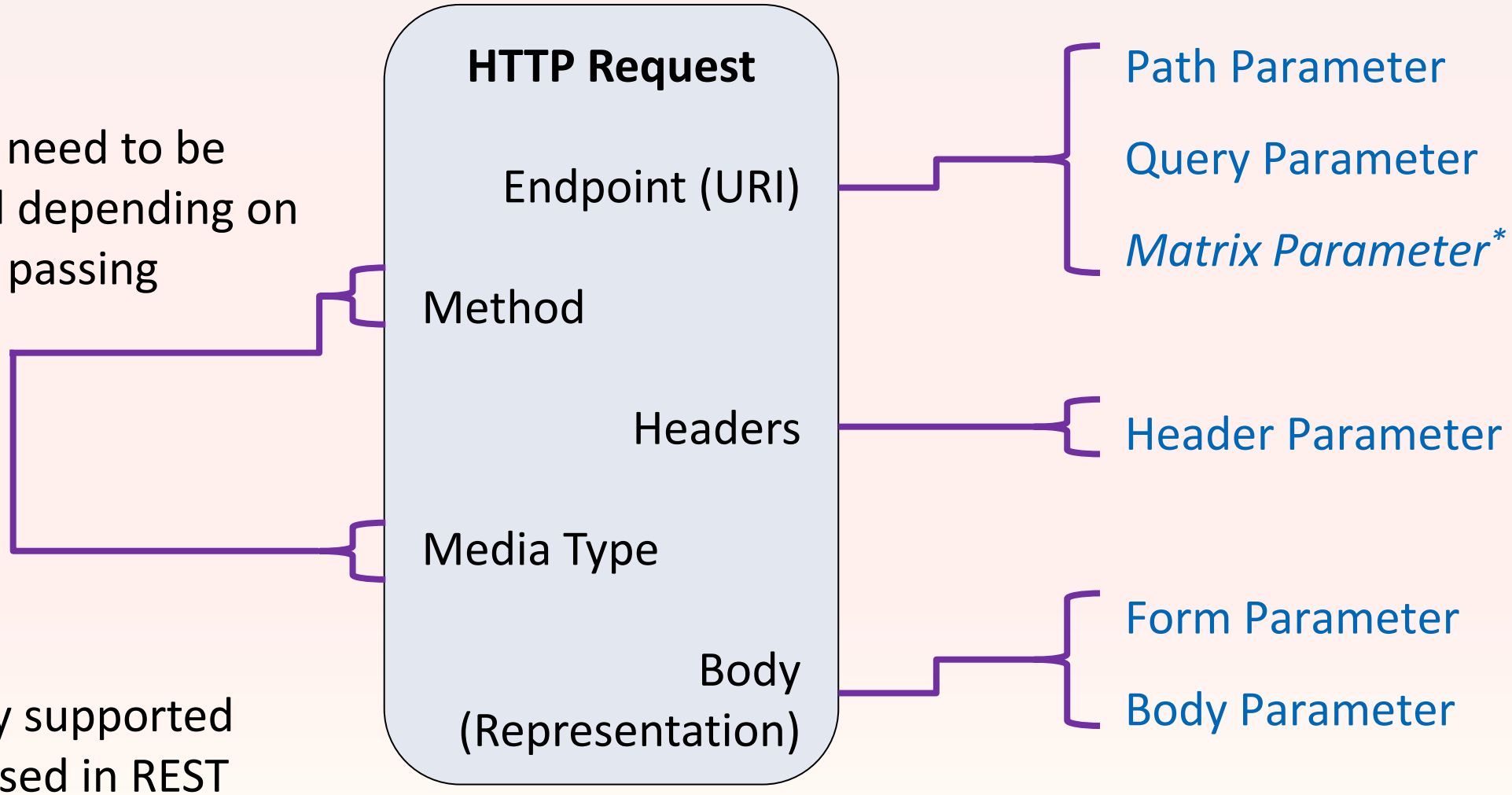
Part of the request describes the format of data and where it is sent.

Other parts can specify parameters that modify the service response.



Parameter Passing: Leveraging HTTP

Note:
These also need to be considered depending on parameter passing method!



* Not widely supported and rarely used in REST

Parameter Passing: Service Example

We have a service for documenting and querying mythical creatures.

The service allows clients to search, update, create and remove entries.

Those operations can be customised using parameters.



The root endpoint for this service will be:

arcanarchive/

Note:

The chosen method to pass parameters will influence the implementation

Parameter Passing: Query Parameter

One of the most common methods – you've likely seen it in your browser.

Customise request by placing key-value pairs in the URI of the form:

www.arcanarchive.me/creatures?type={option}&attribute={option}

www.arcanarchive.me/creatures?type=dragon&attribute=colour

www.arcanarchive.me/creatures?type=unicorn&attribute=magicPower

Parameter Passing: Path Parameters

Used to specify resources or sub-resources directly in the URI.

Allows the service to reuse a single endpoint to support multiple options:

www.archanaarcive.me/creatures/{type}/{attribute}

Client A could send a GET request for a specific resource:

www.archanaarcive.me/creatures/dragon/colour

Client B could send a GET request for a different resource:

www.archanaarcive.me/creatures/unicorn/magicPower

Parameter Passing: Header and Body Parameters

- Header parameters are Key: Value data stored in the HTTP header.
- Some headers have standard names, but you can also create your own!
- Body parameters are anything that is stored in the HTTP body.
- It can be plain text, xml, json or some more complex data

```
GET creatures/ HTTP/1.1
Host: www.arcanarchive.me
Content-Type: text/plain
Cookie: sessionID=secret12345
Custom-Header: someUsefulValue

Hello! Just some plain text here…
```

Parameter Passing: Form Parameter

- Uses same *key=value* syntax as Query Parameters.
- But they're passed in the HTTP request body, not in the URI.
- The Content-Type header defines the data format of the HTTP body.
- Thus, form parameters have their own specific Content-Type.

```
GET creatures/ HTTP/1.1
Host: www.arcanarchive.me
Content-Type: application/x-www-form-urlencoded


creatures?type=dragon&attribute=colour
```

DEMO

Implementing Parameter Passing in Spring Boot

Parameter Passing: General Conventions

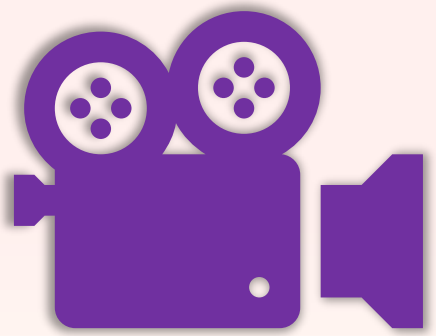
Method	Use when	Avoid when
Path	• Referring to a specific resource • Service partitioned into hierarchy	• Completely optional filters • Arbitrary types of input
Query	• Optional filters on specific resources • Pagination or sorting	• Complex, structured data • Sensitive information
Form	• Supporting legacy HTML form submission	• Complex, structure data
Header	• Sending metadata (content types, etc.) • Sending authentication tokens	• Complex or application level data
Body	• Sending objects or complex data • Creating or updating resources	• Sending GET requests • Sending simple data for filters

Media Types: What's Happening?

Thus far, we haven't been coerced into configuring responses.

- Java Objects are automatically converted to JSON objects
- Plaintext magically appears in the HTTP response body
- Collections are automatically serialised into a list of JSON objects

… and vice versa!



How?

- Spring has been doing some heavy lifting for us.

Media Types: HTTP Headers

Central to data serialisation and deserialisation are the HTTP headers:

Content-Type

- Defines the media (MIME) type of the data in the HTTP body

Accept

- Defines the media (MIME) type the sender is capable of understanding

Media type definitions are straightforward: `type/subtype`

`text/plain`, `application/json`, `text/json`, `application/xml`,

etc…

Media Types: Resource Method

Assume a client request gets routed to the following resource method:

```
@GetMapping("/{index}")
public Iris getIrisByIndex(@PathVariable int index) {
      return data.get(index);
}
```

How does the client get:

- a structured JSON response?
- formatted data describing an Iris?

Media Types: “Automatic” Steps

```
@GetMapping("/{index}")
public Iris getIrisByIndex(@PathVariable int index) {
      return data.get(index);
}
```

Spring Boot:

1. Checks the media type in the Accept header of the incoming request
2. Finds an appropriate `HttpMessageConverter` for that media type
3. Deserialises the object using the Iris class getters and setters and a data-processing library called Jackson

Responses: Beyond the Default

So, we can define how:

- Parameters are passed by the client to the service
- HTTP bodies can be constructed and interpreted when using objects
- Errors can be thrown and automatically return valid HTTP responses

- But what if we want to customise responses beyond that?
- Why haven’t we needed to manually set HTTP status codes?
- Why do all thrown exceptions look the same to the client?

Responses: Customising

Spring supports the `ResponseEntity<T>` class.

Allows us to build HTTP responses in segments, customising:

- Headers
- Response Codes
- Body



Wrapping the data that would have been automatically returned.

DEMO

The ResponseEntity in Spring Boot